Python Project

« What factors are associated with happiness in life? »

Higher School of Economics

Faculty of Computer Science

Kim Jae-Wook

Table of contents

0. Basic Preparation **2**
Packages and Data Loading . . . 2
About Data "YPS.csv" . . . 2

1. Data Overview : Understadning given Dataset "YPS.csv" **3**
1 - 1. Display all column names of table in clear readable format . . . 3
1 - 2. Data Summary Table . . . 4

2. Formulate Research Question + Select Variables for analysis **8**
2 - 1. Research Question : « What factors are associated with happiness in life? » . . . 8
2 - 2. Selected Variables . . . 8
2 - 3. Control Variables : Age and Gender (Must be included) . . . 8
2 - 4. Condition Satisfied : " Create subset with all variables that you need " . . . 8
2 - 5. Information about ALL selected variables Before Data Preprocessing . . . 8
2 - 6. Data Preprocessing : Handling Missing Values of Numeric and Categorical Data . 10
2 - 7. Information about ALL selected variables After Data Preprocessing . . . 12

3. Exploratory Data Analysis (EDA) **13**
3 - 1. Providing basic descriptive statistics on Numeric Variables . . . 13
3 - 2. Providing basic descriptive statistics on Categorical Variable . . . 15
3 - 3. Visualization of Distributions . . . 16
3 - 4. Associations among numeric variables (Spearman correlation) . . . 29
3 - 5. Gender differences in personal traits and happiness . . . 31

4. Modeling and Comparison **42**
4 - 1. Data Splitting . . . 42
4 - 2. Preprocessing Pipeline Construction . . . 43
4 - 3. Implementing THREE Regression Models (Linear, Huber, Random Forest) . . . 43
4 - 4. Best Model Selection . . . 46

5. Interpretation and Limitations **47**
5 - 1. Basic concepts for Interpretation . . . 48
5 - 2. Interpretation for Coefficients Table . . . 48
5 - 3. Interpretation about Strongest Positive Association . . . 48
5 - 4. Interpretation about Strongest Negative Association . . . 49
5 - 5. Interpretation about Neutral Association . . . 50
5 - 6. Effects of Features on Predicted Happiness using Huber Regression . . . 50
5 - 7. Interpretation of Above Bar Plot . . . 51
5 - 8. Residuals vs Predicted on Test Set using Huber Regression . . . 52
5 - 9. Interpretation of Above Residual Plot . . . 52
5 - 10. Overall Conclusion . . . 53
5 - 11. Model-related Limitations . . . 53
5 - 12. Dataset-related limitations . . . 54

6. Epliogue : Thoughts I Had During Project **54**

0. Basic Preparation

Packages and Data Loading

```
# Import necessary Packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from scipy import stats

# Data Loading for project
CSV_PATH = "YPS.csv"  # Set relative file path to CSV dataset (file name is "YPS.csv")
df = pd.read_csv(CSV_PATH)    # Read CSV file into DataFrame named 'df'
```

About Data “YPS.csv”

- YPS stands for Young People Survey.
- Dataset contains 1010 rows and 150 columns. (First row is column names not counted in 1010 row data, so total number of rows is 1011, if column names are included)

File Source

- The file (YPS.csv) can be downloaded from Google Drive folder (https://drive.google.com/drive/folders/1W2rcFCRDWaCeysVgU7Al6KO-HPi11Rkp) provided by lecturer.
- It can also be downloaded from https://www.kaggle.com/datasets/miroslavsabo/young-people-survey?resource=download&select=responses.csv

About Basic Packages

- pandas : for data manipulation and analysis handling dataframe structures and file input and output
- numpy : for numerical computing performing vectorized operations and linear algebra with n dimensional arrays
- matplotlib.pyplot : for data visualization creating and customizing 2D plots and charts
- pathlib.Path : for file system management providing object oriented interface for path operations
- scipy.stats : for statistical analysis offering probability distributions and hypothesis testing functions

1. Data Overview : Understadning given Dataset “YPS.csv”

1 - 1. Display all column names of table in clear readable format

⇒ In order to identify all variables that can serve as potential subjects for analysis.

⇒ Why is it needed to identify all variables? Because it helps come up with research question.

```
cols = pd.read_csv("YPS.csv", nrows=0).columns.tolist() # Read header only
print(f"Count = {len(cols)}")  # Print count of columns

PER_LINE = 5       # Number of Column name in one line
SEP = " | "        # Blank for both sides of |

# Loop by step PER_LINE  # Print chunked names joined by separator for visibility
for i in range(0, len(cols), PER_LINE):
   print(SEP.join(cols[i:i+PER_LINE]))
```

Count = 150
Music | Slow songs or fast songs | Dance | Folk | Country
Classical music | Musical | Pop | Rock | Metal or Hardrock
Punk | Hiphop, Rap | Reggae, Ska | Swing, Jazz | Rock n roll
Alternative | Latino | Techno, Trance | Opera | Movies
Horror | Thriller | Comedy | Romantic | Sci-fi
War | Fantasy/Fairy tales | Animated | Documentary | Western
Action | History | Psychology | Politics | Mathematics
Physics | Internet | PC | Economy Management | Biology
Chemistry | Reading | Geography | Foreign languages | Medicine
Law | Cars | Art exhibitions | Religion | Countryside, outdoors
Dancing | Musical instruments | Writing | Passive sport | Active sport
Gardening | Celebrities | Shopping | Science and technology | Theatre
Fun with friends | Adrenaline sports | Pets | Flying | Storm
Darkness | Heights | Spiders | Snakes | Rats
Ageing | Dangerous dogs | Fear of public speaking | Smoking | Alcohol
Healthy eating | Daily events | Prioritising workload | Writing notes | Workaholism
Thinking ahead | Final judgement | Reliability | Keeping promises | Loss of interest
Friends versus money | Funniness | Fake | Criminal damage | Decision making
Elections | Self-criticism | Judgment calls | Hypochondria | Empathy
Eating to survive | Giving | Compassion to animals | Borrowed stuff | Loneliness
Cheating in school | Health | Changing the past | God | Dreams
Charity | Number of friends | Punctuality | Lying | Waiting
New environment | Mood swings | Appearence and gestures | Socializing | Achievements
Responding to a serious letter | Children | Assertiveness | Getting angry | Knowing the right people
Public speaking | Unpopularity | Life struggles | Happiness in life | Energy levels
Small - big dogs | Personality | Finding lost valuables | Getting up | Interests or hobbies
Parents' advice | Questionnaires or polls | Internet usage | Finances | Shopping centres
Branded clothing | Entertainment spending | Spending on looks | Spending on gadgets | Spending on healthy eating
Age | Height | Weight | Number of siblings | Gender
Left - right handed | Education | Only child | Village - town | House - block of flats

1 - 2. Data Summary Table

Create and Display Data Summary Table

⇒ So that all the information about variables can be seen at one sight.

```
from IPython.display import HTML

# get list of numeric column names for later numeric summaries
numeric_cols = df.select_dtypes(include="number").columns.tolist()

def erase_dot_zero(x):
    """Convert float to int (keeps pd.NA as is)"""
    if pd.isna(x):
        return pd.NA
    try:
        return int(round(x))
    except Exception:
        return pd.NA

summary = pd.DataFrame({
    "Variable": [str(c)[:7] for c in df.columns],
    "Type": ["numeric" if c in numeric_cols else "string" for c in df.columns],
    "NoMiss": df.notna().sum().values,
    "Miss": df.isna().sum().values,
    "Min": [erase_dot_zero(df[c].min(skipna=True)) if c in numeric_cols else pd.NA for c in df.columns],
    "Max": [erase_dot_zero(df[c].max(skipna=True)) if c in numeric_cols else pd.NA for c in df.columns],
})
summary = summary.fillna("NA")  # Use "NA" string for better alignment in table than default setting of <NA>

# Split the DataFrame into two columns (left/right) for display
TOTAL = len(summary)   # Use ceiling division to split rows evenly.
# Left side may have one extra row if TOTAL is odd.
half = (TOTAL + 1) // 2

# Splits summary table into two halves and resets indices for alignment.
# Then joins both halves side by side    # To make two-column layout for clearer display.

left = summary.iloc[:half].reset_index(drop=True)  # left half of the summary
right = summary.iloc[half:].reset_index(drop=True) # right half of the summary
merged = pd.concat([left, right], axis=1) # place halves side by side
```

After many trials decided to place table on both left and right sides to fill entire page.

(Reason 1) If I put table based on default setting (Variable, NoMiss, Miss, Min, Max each of those appears once), then space on right side was too large when table placed in single column.

(Reason 2) This layout allows efficient overview of many variables at a glance.

Also I decided to use HTML table format for better visuality among all methods tested.

```
# Display as HTML table with clear and well-alligned borders
HTML(merged.to_html(index=False, border=1))
```

Variable	Type	NoMiss	Miss	Min	Max	Variable	Type	NoMiss	Miss	Min	Max
Music	numeric	1007	3	1	5	Healthy	numeric	1007	3	1	5
Slow so	numeric	1008	2	1	5	Daily e	numeric	1003	7	1	5
Dance	numeric	1006	4	1	5	Priorit	numeric	1005	5	1	5
Folk	numeric	1005	5	1	5	Writing	numeric	1007	3	1	5
Country	numeric	1005	5	1	5	Workaho	numeric	1005	5	1	5
Classic	numeric	1003	7	1	5	Thinkin	numeric	1007	3	1	5
Musical	numeric	1008	2	1	5	Final j	numeric	1003	7	1	5
Pop	numeric	1007	3	1	5	Reliabi	numeric	1006	4	1	5
Rock	numeric	1004	6	1	5	Keeping	numeric	1009	1	1	5
Metal o	numeric	1007	3	1	5	Loss of	numeric	1006	4	1	5
Punk	numeric	1002	8	1	5	Friends	numeric	1004	6	1	5
Hiphop,	numeric	1006	4	1	5	Funnine	numeric	1006	4	1	5
Reggae,	numeric	1003	7	1	5	Fake	numeric	1009	1	1	5
Swing,	numeric	1004	6	1	5	Crimina	numeric	1003	7	1	5
Rock n	numeric	1003	7	1	5	Decisio	numeric	1006	4	1	5
Alterna	numeric	1003	7	1	5	Electio	numeric	1007	3	1	5
Latino	numeric	1002	8	1	5	Self-cr	numeric	1005	5	1	5
Techno,	numeric	1003	7	1	5	Judgmen	numeric	1006	4	1	5
Opera	numeric	1009	1	1	5	Hypocho	numeric	1006	4	1	5
Movies	numeric	1004	6	1	5	Empathy	numeric	1005	5	1	5
Horror	numeric	1008	2	1	5	Eating	numeric	1010	0	1	5
Thrille	numeric	1009	1	1	5	Giving	numeric	1004	6	1	5
Comedy	numeric	1007	3	1	5	Compass	numeric	1003	7	1	5
Romanti	numeric	1007	3	1	5	Borrowe	numeric	1008	2	1	5
Sci-fi	numeric	1008	2	1	5	Lonelin	numeric	1009	1	1	5
War	numeric	1008	2	1	5	Cheatin	numeric	1006	4	1	5
Fantasy	numeric	1007	3	1	5	Health	numeric	1009	1	1	5
Animate	numeric	1007	3	1	5	Changin	numeric	1008	2	1	5
Documen	numeric	1002	8	1	5	God	numeric	1008	2	1	5
Western	numeric	1006	4	1	5	Dreams	numeric	1010	0	1	5
Action	numeric	1008	2	1	5	Charity	numeric	1007	3	1	5
History	numeric	1008	2	1	5	Number	numeric	1010	0	1	5
Psychol	numeric	1005	5	1	5	Punctua	string	1008	2	NA	NA
Politic	numeric	1009	1	1	5	Lying	string	1008	2	NA	NA
Mathema	numeric	1007	3	1	5	Waiting	numeric	1007	3	1	5
Physics	numeric	1007	3	1	5	New env	numeric	1008	2	1	5
Interne	numeric	1006	4	1	5	Mood sw	numeric	1006	4	1	5
PC	numeric	1004	6	1	5	Appeare	numeric	1007	3	1	5
Economy	numeric	1005	5	1	5	Sociali	numeric	1005	5	1	5
Biology	numeric	1004	6	1	5	Achieve	numeric	1008	2	1	5

Variable	Type	NoMiss	Miss	Min	Max	Variable	Type	NoMiss	Miss	Min	Max
Chemist	numeric	1000	10	1	5	Respond	numeric	1004	6	1	5
Reading	numeric	1004	6	1	5	Childre	numeric	1006	4	1	5
Geograp	numeric	1001	9	1	5	Asserti	numeric	1008	2	1	5
Foreign	numeric	1005	5	1	5	Getting	numeric	1006	4	1	5
Medicin	numeric	1005	5	1	5	Knowing	numeric	1008	2	1	5
Law	numeric	1009	1	1	5	Public	numeric	1008	2	1	5
Cars	numeric	1006	4	1	5	Unpopul	numeric	1007	3	1	5
Art exh	numeric	1004	6	1	5	Life st	numeric	1007	3	1	5
Religio	numeric	1007	3	1	5	Happine	numeric	1006	4	1	5
Country	numeric	1003	7	1	5	Energy	numeric	1005	5	1	5
Dancing	numeric	1007	3	1	5	Small -	numeric	1006	4	1	5
Musical	numeric	1009	1	1	5	Persona	numeric	1006	4	1	5
Writing	numeric	1004	6	1	5	Finding	numeric	1006	4	1	5
Passive	numeric	995	15	1	5	Getting	numeric	1005	5	1	5
Active	numeric	1006	4	1	5	Interes	numeric	1007	3	1	5
Gardeni	numeric	1003	7	1	5	Parents	numeric	1008	2	1	5
Celebri	numeric	1008	2	1	5	Questio	numeric	1006	4	1	5
Shoppin	numeric	1008	2	1	5	Interne	string	1010	0	NA	NA
Science	numeric	1004	6	1	5	Finance	numeric	1007	3	1	5
Theatre	numeric	1002	8	1	5	Shoppin	numeric	1008	2	1	5
Fun wit	numeric	1006	4	2	5	Branded	numeric	1008	2	1	5
Adrenal	numeric	1007	3	1	5	Enterta	numeric	1007	3	1	5
Pets	numeric	1006	4	1	5	Spendin	numeric	1007	3	1	5
Flying	numeric	1007	3	1	5	Spendin	numeric	1010	0	1	5
Storm	numeric	1009	1	1	5	Spendin	numeric	1008	2	1	5
Darknes	numeric	1008	2	1	5	Age	numeric	1003	7	15	30
Heights	numeric	1007	3	1	5	Height	numeric	990	20	62	203
Spiders	numeric	1005	5	1	5	Weight	numeric	990	20	41	165
Snakes	numeric	1010	0	1	5	Number	numeric	1004	6	0	10
Rats	numeric	1007	3	1	5	Gender	string	1004	6	NA	NA
Ageing	numeric	1009	1	1	5	Left -	string	1007	3	NA	NA
Dangero	numeric	1009	1	1	5	Educati	string	1009	1	NA	NA
Fear of	numeric	1009	1	1	5	Only ch	string	1008	2	NA	NA
Smoking	string	1002	8	NA	NA	Village	string	1006	4	NA	NA
Alcohol	string	1005	5	NA	NA	House -	string	1006	4	NA	NA

(!!!) Special Comment from author about Rendering Tables on this report

This table is designed to display beautifully. That is, all column names and their corresponding entries are neatly aligned in rows and columns. However, when rendering HTML or PDF output with Quarto (.qmd file using Pandoc engine), the table alignment may occasionally break.

Please, Do not worry !

Re-rendering the document in Quarto will restore table to its clean and organized appearance.

Overall Explanation of Table Contents

- This table includes the columns Variable, Type, NoMiss, Miss, Min, and Max.
- Most variables are numeric, and their values range from 1 to 5, representing a simple five-point rating scale.
- Missing values are few, and the number of valid responses is around 1000 for most variables.
- Some variables are text type, such as Gender, Left-, Educati, Only chi, Village, and House-.
- These text variables show NA in the Min and Max columns.
- Continuous demographic variables such as Age, Height, and Weight are also included.
- Variable names are shortened to the first seven characters, so some names appear partly cut off.
- The table is split into two parts and joined side by side, allowing more data to be shown on one page.
- Variables are grouped by theme, including music genres, movie genres, personality traits, and fears or dislikes.
- The summary shows minimum and maximum values only for numeric variables, while both missing and non-missing counts are given for all.
- Overall, the dataset shows high completeness, low missingness, and consistent value ranges, making it well suited for data exploration and analysis.

2. Formulate Research Question + Select Variables for analysis

2 - 1. Research Question : « What factors are associated with happiness in life? »

```
# Define columns (use exact names from CSV for analysis)
target = "Happiness in life"
features_core = [
    "Health", "Energy levels", "Active sport", "Healthy eating",
    "Number of friends", "Socializing", "Loneliness", "Mood swings",
]
controls = ["Age", "Gender"]

selected = [target] + features_core + controls

data = df[selected].copy()  # Build subset (keep missing at this moment)
```

2 - 2. Selected Variables

⇒ “Health”, “Energy levels”, “Active sport”, “Healthy eating”, “Number of friends”, “Socializing”, “Loneliness”, “Mood swings”

2 - 3. Control Variables : Age and Gender (Must be included)

⇒ Age and Gender variables are included as control which is given condition from professor for this project.

2 - 4. Condition Satisfied : “ Create subset with all variables that you need ”

⇒ This is because subset consist of features named **selected** among above codes includes all required variables such as target variable, core features, and control variables Age and Gender.

2 - 5. Information about ALL selected variables Before Data Preprocessing

⇒ Create Summary Table about ALL selected variables BEFORE Data Preprocessing.

```
# Coerce numeric-like to numeric
numeric_like = features_core + ["Age"]
for c in numeric_like:
    data[c] = pd.to_numeric(data[c], errors="coerce")

# prepare missingness summary before imputation
numeric_cols = data.select_dtypes(include="number").columns.tolist()

def erase_dot_zero(x):
    """convert float to int; keep NA"""
    if pd.isna(x):
        return pd.NA
    try:
```

```
        return int(round(x))
    except Exception:
        return pd.NA

summary = pd.DataFrame({
    "Variable": [str(c)[:7] for c in data.columns],
    "Type": ["numeric" if c in numeric_cols else "string" for c in data.columns],
    "NoMiss": data.notna().sum().values,
    "Miss": data.isna().sum().values,
    "Min": [erase_dot_zero(data[c].min(skipna=True)) if c in numeric_cols else pd.NA for c in data.columns],
    "Max": [erase_dot_zero(data[c].max(skipna=True)) if c in numeric_cols else pd.NA for c in data.columns],
}).fillna("NA")

# split summary into two columns for display
TOTAL = len(summary)
half = (TOTAL + 1) // 2
left = summary.iloc[:half].reset_index(drop=True)
right = summary.iloc[half:].reset_index(drop=True)

# force integer display for NoMiss and Miss (avoid .0)
for part in (left, right):
    for col in ("NoMiss", "Miss"):
        part[col] = pd.to_numeric(part[col], errors="coerce").astype("Int64")

merged = pd.concat([left, right], axis=1)

# ensure hyphen for all missing including <NA> from Int64
merged_display = merged.astype(object).where(merged.notna(), "-")

# render HTML table
HTML(merged_display.to_html(index=False, border=1))
```

Variable	Type	NoMiss	Miss	Min	Max	Variable	Type	NoMiss	Miss	Min	Max
Happine	numeric	1006	4	1	5	Sociali	numeric	1005	5	1	5
Health	numeric	1009	1	1	5	Lonelin	numeric	1009	1	1	5
Energy	numeric	1005	5	1	5	Mood sw	numeric	1006	4	1	5
Active	numeric	1006	4	1	5	Age	numeric	1003	7	15	30
Healthy	numeric	1007	3	1	5	Gender	string	1004	6	NA	NA
Number	numeric	1010	0	1	5	-	-	-	-	-	-

Based on above Table about all selected variables, Data Preprocessing will be conducted from next page.

2 - 6. Data Preprocessing : Handling Missing Values of Numeric and Categorical Data

```
data = data.dropna(subset=[target])   # Drop rows with target ('Happiness in life') missing

# Define numeric columns (core features + Age + target) and categorical columns (Gender)
num_cols = [c for c in (features_core + ["Age", target]) if c in data.columns]
cat_cols = [c for c in ["Gender"] if c in data.columns]

# How to deal with Missing Values : Basic imputation
# numeric: median      # categorical (Gender): most frequent
num_cols_no_target = [c for c in num_cols if c != target]   # Keep numeric predictors only
for c in num_cols_no_target:
    if data[c].isna().any():
        med = data[c].median()
        data[c] = data[c].fillna(med)

for c in cat_cols:
    if data[c].isna().any():
        mode = data[c].mode(dropna=True)
        fill_val = mode.iloc[0] if len(mode) > 0 else "Unknown"
        data[c] = data[c].fillna(fill_val)

# Simple summary table for cleaned subset
print(f"Number of observations after handling missing values : {len(data)}")
```

Number of observations after handling missing values : 1006

```
def format_summary_value(x):
    """Format numbers for the summary table."""
    if pd.isna(x):
        return pd.NA
    try:
        if np.isclose(x, round(x)):   # If it's integer or float that is whole number
            return int(round(x))
        # Otherwise, it's float (like median 2.5), so format it
        return f"{x:.1f}"
    except Exception:
        return pd.NA

# Get numeric columns from DataFrame
numeric_cols_summary = data.select_dtypes(include="number").columns.tolist()

# Write Min/Max lists beforehand for suitable line breaks when exporting PDF
mins = [
    format_summary_value(data[c].min(skipna=True))
```

```
    if c in numeric_cols_summary
    else pd.NA
    for c in data.columns
]

maxs = [
    format_summary_value(data[c].max(skipna=True))
    if c in numeric_cols_summary
    else pd.NA
    for c in data.columns
]

summary = pd.DataFrame({
    # Increased length for "Happiness in life" etc.
    "Variable": [str(c)[:17] for c in data.columns],
    "Type": ["numeric" if c in numeric_cols_summary else "string" for c in data.columns],
    "NoMiss": data.notna().sum().values,
    "Miss": data.isna().sum().values,
    "Min": mins,
    "Max": maxs,
})
summary = summary.fillna("NA")

# Apply CSS styling for consistent left alignment to fix alignment issue
styles = [
    # Style for header cells (th)
    dict(selector="th", props=[
        ("text-align", "left"),
        ("font-weight", "bold"),
        ("border", "1px solid black")
    ]),
    # Style for data cells (td)
    dict(selector="td", props=[
        ("text-align", "left"),
        ("border", "1px solid black")
    ])
]
# Create styled DataFrame
styled_summary = (
    summary.style
    .set_table_styles(styles)
    .set_table_attributes('style="border-collapse: collapse; border: 1px solid black;"')
    .hide(axis="index")
    .format(na_rep="-")
)
```

Data Preprocessing Summary

- Remove rows with missing target (Happiness in life)
- Identify numeric and categorical columns for proper handling
- Replace missing numeric values using median value
- Replace missing categorical values using most frequent value
- Check number of remaining observations after cleaning
- Generate summary table showing variable type, missing counts, min and max values
- Format numeric values for clear presentation
- Apply HTML and CSS styling for clean table layout
- Obtain well-prepared dataset and readable summary table for next analysis steps

2 - 7. Information about ALL selected variables After Data Preprocessing

```
# Display styled HTML
display(HTML(styled_summary.to_html()))
```

Table 3

Variable	Type	NoMiss	Miss	Min	Max
Happiness in life	numeric	1006	0	1	5
Health	numeric	1006	0	1	5
Energy levels	numeric	1006	0	1	5
Active sport	numeric	1006	0	1	5
Healthy eating	numeric	1006	0	1	5
Number of friends	numeric	1006	0	1	5
Socializing	numeric	1006	0	1	5
Loneliness	numeric	1006	0	1	5
Mood swings	numeric	1006	0	1	5
Age	numeric	1006	0	15	30
Gender	string	1006	0	NA	NA

Comments about above Table

- All selected variables have 1006 valid observations with no missing values.
- Most variables are numeric on 1–5 Likert scale including target Happiness in life and eight predictors such as Health, Energy levels, Active sport, Healthy eating, Number of friends, Socializing, Loneliness, and Mood swings.
- Control variable Age ranges from 15 to 30.
- Gender is string variable used as categorical feature.

3. Exploratory Data Analysis (EDA)

On this chapter, Exploratory Analysis of selected variables is conducted based on requirements of project.

```
# Import visualization and stats libraries
import seaborn as sns
from IPython.display import display

# Set plot style for consistency
sns.set_theme(style="whitegrid")
```

3 - 1. Providing basic descriptive statistics on Numeric Variables

```
# Descriptive Statistics (Numeric)
_desc = data[num_cols].describe().T  # original computation kept

# post-process: drop ".0" only for integer-like summary columns
import pandas as pd
def _drop_dot_zero(v):
    if pd.isna(v):
        return v
    try:
        f = float(v)
    except Exception:
        return v
    return int(round(f)) if abs(f - round(f)) < 1e-9 else v

for _c in ["count", "min", "25%", "50%", "75%", "max"]:
    if _c in _desc.columns:
        _desc[_c] = _desc[_c].apply(_drop_dot_zero)
```

```
print("\n      Table for Descriptive Statistics ( Numeric Variables )")
display(_desc)
```

Table for Descriptive Statistics (Numeric Variables)

	count	mean	std	min	25%	50%	75%	max
Health	1006	3.247515	1.074984	1	3	3	4	5
Energy levels	1006	3.637177	0.996796	1	3	4	4	5
Active sport	1006	3.291252	1.501463	1	2	3	5	5
Healthy eating	1006	3.028827	0.934770	1	3	3	4	5
Number of friends	1006	3.343936	1.056135	1	3	3	4	5
Socializing	1006	3.159046	1.090234	1	2	3	4	5
Loneliness	1006	2.883698	1.130928	1	2	3	4	5
Mood swings	1006	3.254473	1.041839	1	3	3	4	5
Age	1006	20.428429	2.806238	15	19	20	22	30
Happiness in life	1006	3.705765	0.824323	1	3	4	4	5

Interpretation of Descriptive Statistics Table

→ Table summarizes numeric variables describing participants' health, lifestyle, and happiness. Each variable has 1006 valid responses. Values measured on 1–5 Likert scale, except Age which is continuous.

- Central tendency : Mean values around 3 indicate moderate levels for most lifestyle factors (Health = 3.25, Energy levels = 3.64).
- Variation : Standard deviations near 1 show individual responses vary but not extremely.
- Distribution shape : Quartiles (25–50–75%) show most answers cluster around 3–4, meaning participants tend to rate themselves slightly positively.
- Age : Average about 20 years, narrow spread (SD = 2.8) implies young homogeneous group.
- Happiness in life : Highest mean = 3.71 shows relatively positive overall life satisfaction.

→ Overall, participants appear healthy, socially active, and moderately happy without extreme differences among lifestyle or mood variables.

3 - 2. Providing basic descriptive statistics on Categorical Variable

```
# Descriptive Statistics (Categorical)
print("\n     Frequency Table for Descriptive Statistics ( Categorical Variable = Gender )")
for col in cat_cols:
    # Used dropna=False to see if any imputed values ('Unknown') exist
    freq = data[col].value_counts(dropna=False)
    prop = data[col].value_counts(normalize=True, dropna=False)
    display(pd.DataFrame({'Count': freq, 'Proportion': prop}))
```

Frequency Table for Descriptive Statistics (Categorical Variable = Gender)

Gender	Count	Proportion
female	597	0.593439
male	409	0.406561

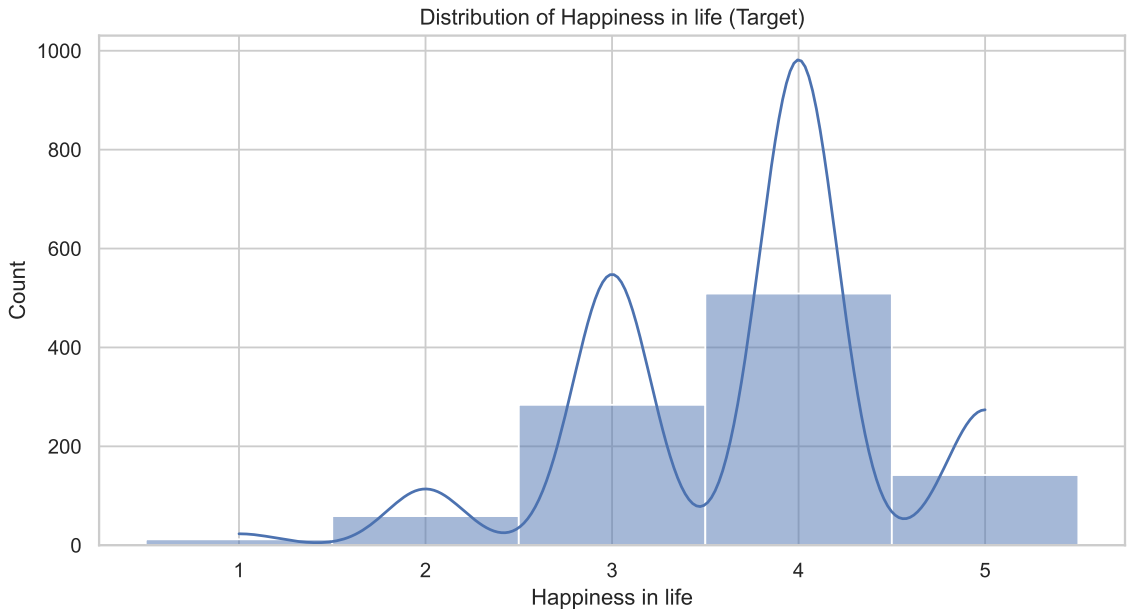
Interpretation of Gender Frequency Table

- Above Table shows gender composition of participants.
- Among 1006 respondents, 597 are female and 409 are male.
- Proportions are 0.593 and 0.407 respectively.
- Majority of participants are female, accounting for about 59 percent of sample, while males make up about 41 percent.
- This indicates slightly unbalanced gender distribution but still both genders sufficiently represented for analysis.

3 - 3. Visualization of Distributions

3 - 3 - 1. Visualization of Target Value Distribution

```
# 1. Distribution Plot (Target: Happiness)
plt.figure(figsize=(10, 5))
sns.histplot(data[target], bins=5, kde=True, discrete=True)
plt.title('Distribution of Happiness in life (Target)')
plt.show()
```



Small Comment for Distribution of Happiness in life

- Distribution of happiness shows clear right skew with majority of participants rating 3 or 4 on 1–5 scale.
- Peak around 4 indicates generally positive life satisfaction.
- Smaller group rated 5, while few participants rated 1 or 2, suggesting low unhappiness rate.

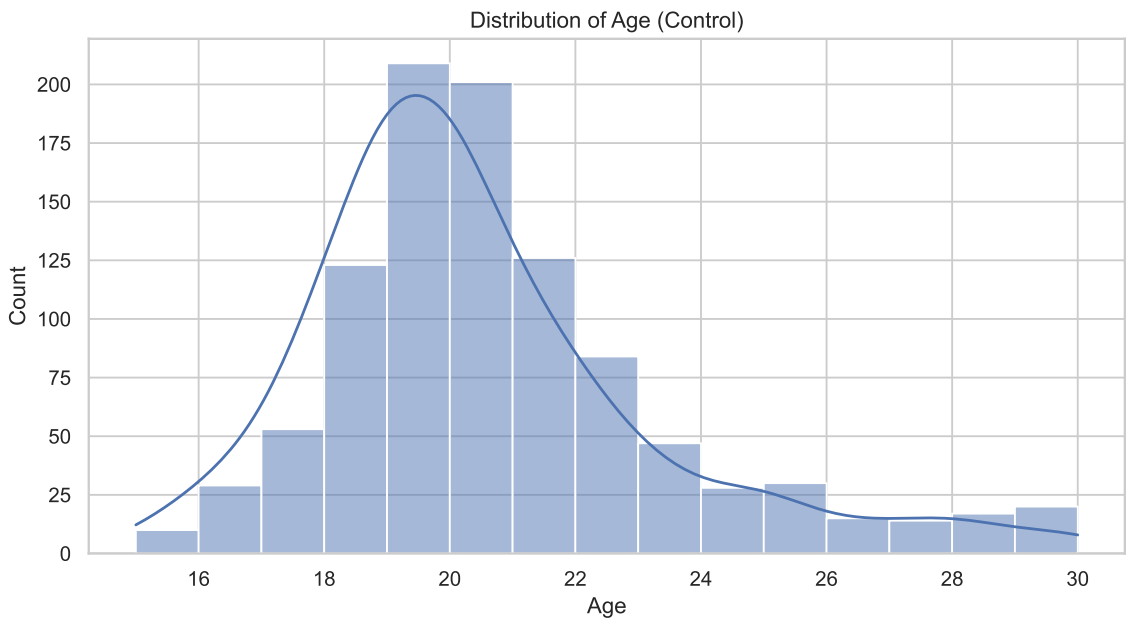
Why does a histogram include both bars and a line?

- **Histogram bars**
  - show count within each value range
  - reveal where observations concentrate

  - useful for quick view of distribution shape
- **Kernel Density Estimate (KDE) line**
  - smooth curve that estimates overall density
  - reduces noise from bin choice and bar jaggedness
  - helps compare shape with bell like curve by eye
  - useful when judging skewness and multimodality

3 - 3 - 2. Visualization of Control Value (Age) Distribution + Check Normality

```
# 2. Distribution Plot (Control: Age)
plt.figure(figsize=(10, 5))
sns.histplot(data['Age'], kde=True, bins=15) # Range is 15-30
plt.title('Distribution of Age (Control)')
plt.show()
```



Small Comment for Distribution of Age

- Age distribution shows approximately normal shape with slight right skew.
- Most participants are between 18 and 22 years old, peaking around 20.
- Few respondents are younger than 17 or older than 25, indicating sample mainly consists of young adults with limited age variation.

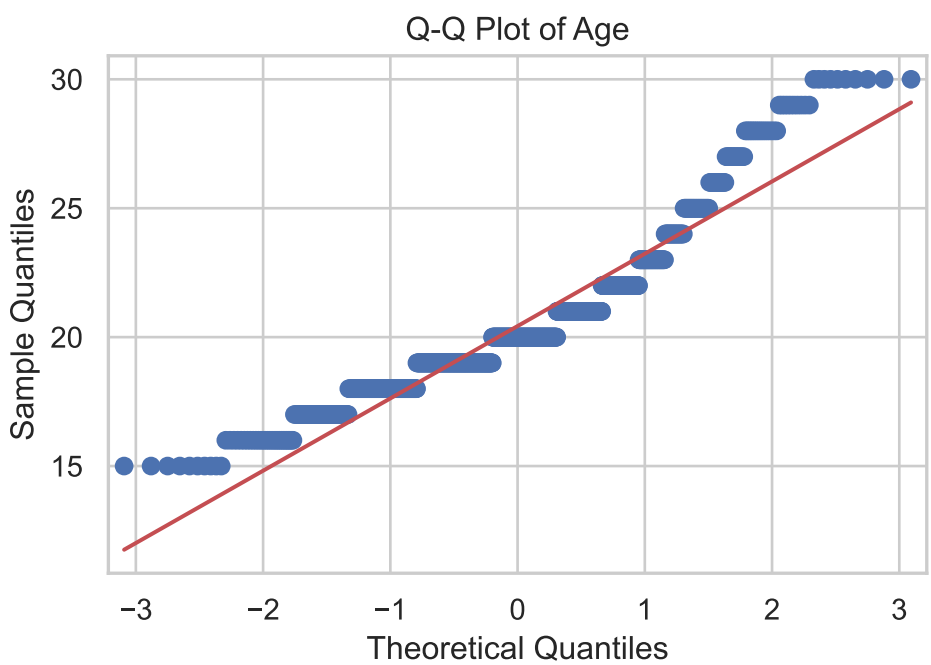
Why only Age has meaningful normality check ?

- Age is continuous numeric so normality concept applies
- Likert variables use discrete ordered scores from 1 to 5 so not continuous
- normality tests assume continuous data so applying to Likert scores is not appropriate

3 - 3 - 3. Check Normality for Control Variable (Age)

```
import statsmodels.api as sm

# Shapiro-Wilk test
age = data['Age'].dropna()
stat, p = stats.shapiro(age)
print(f"Shapiro-Wilk W={stat:.3f}, p-value={p:.4f}")
print("Normality rejected" if p < 0.05 else "Normality not rejected")

# Q-Q plot
sm.qqplot(age, line='s')
plt.title("Q-Q Plot of Age")
plt.show()
```

Shapiro-Wilk W=0.907, p-value=0.0000
Normality rejected



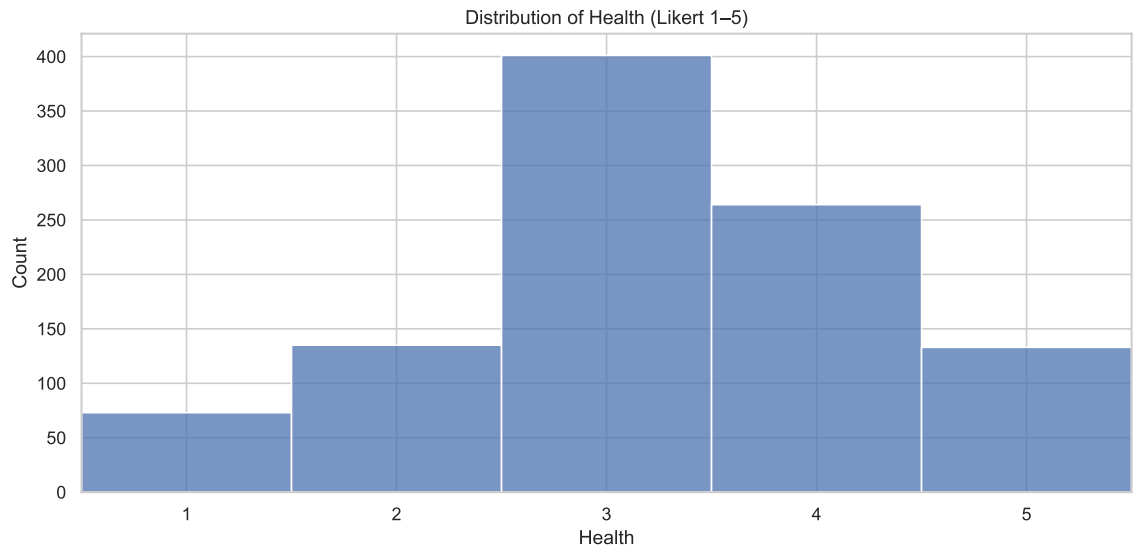
Normality check of Age using Q–Q plot

- Plot shows Q–Q comparison of Age against theoretical normal quantiles.
- Points follow diagonal fairly well through middle, indicating approximate normality in central range.
- Clear step pattern appears because Age takes integer values, so quantiles cluster at discrete levels.
- Right tail bends above diagonal and flattens near top, suggesting slightly heavier upper tail or mild positive skew and some ceiling at older ages.
- Left tail sits a bit below diagonal, consistent with lighter tail at younger ages.
- Overall, Age is roughly normal in middle but deviates in tails due to discreteness and skew.
- For models assuming normal residuals only, this pattern is acceptable if residuals look well behaved; for methods sensitive to tail behavior, consider transforms or robust estimators.

3 - 3 - 4. Visualization of key feature (Health) Distribution

```
# Likert 1–5 bins and ticks
likert_bins = np.arange(0.5, 5.6, 1.0)
likert_ticks = [1, 2, 3, 4, 5]
```

```
# -- Distribution : Health --
col = "Health"
s = data[col].dropna()
plt.figure(figsize=(10, 5))
sns.histplot(s, bins=likert_bins, discrete=True, kde=False, stat="count")  # use stat="percent" for %
plt.xticks(likert_ticks)
plt.xlim(0.5, 5.5)
plt.xlabel(col)
plt.ylabel("Count")
plt.title(f"Distribution of {col} (Likert 1–5)")
plt.tight_layout()
plt.show()
```

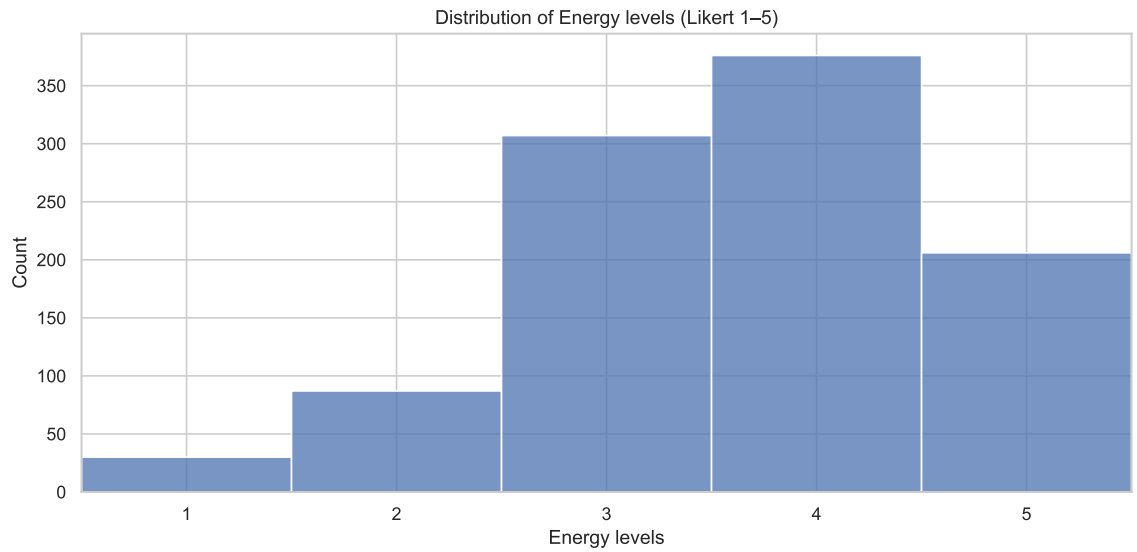


Small Comment for Distribution of Health

- Distribution of self-rated health shows concentration around middle value 3, with secondary peak at 4.
- Responses of 1 or 2 are less frequent, indicating majority perceive their health as average to good.
- Only small portion rates 5, suggesting few participants consider their health excellent.

3 - 3 - 5. Visualization of key feature (Energy levels) Distribution

```
# -- Distribution : Energy levels --
col = "Energy levels"
s = data[col].dropna()
plt.figure(figsize=(10, 5))
sns.histplot(s, bins=likert_bins, discrete=True, kde=False, stat="count")  # use stat="percent" for %
plt.xticks(likert_ticks)
plt.xlim(0.5, 5.5)
plt.xlabel(col)
plt.ylabel("Count")
plt.title(f"Distribution of {col} (Likert 1–5)")
plt.tight_layout()
plt.show()
```

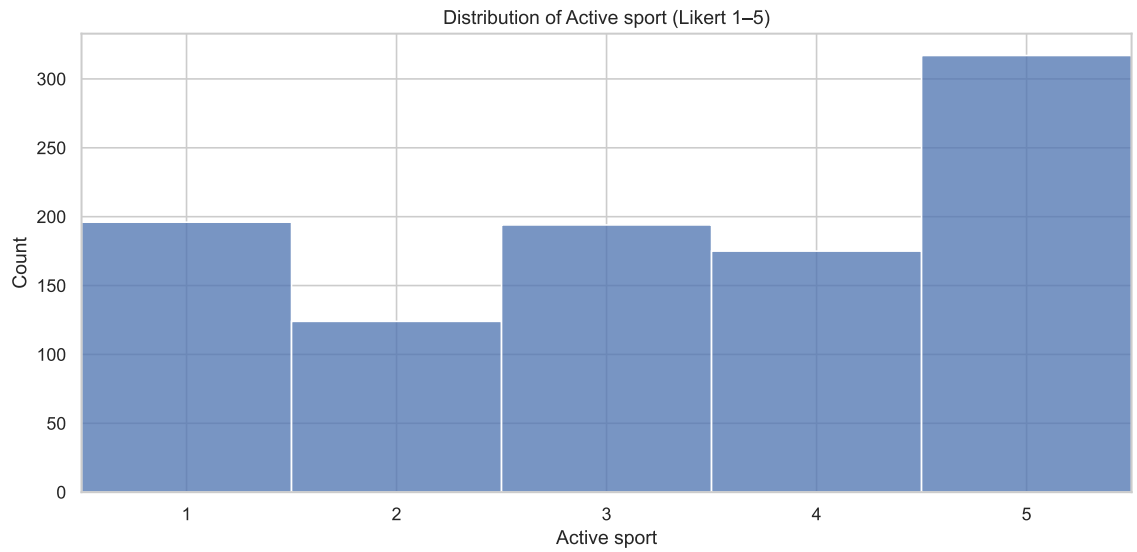


Small Comment for Distribution of Energy levels

- Distribution of energy levels shows clear positive tendency.
- Most participants rated 3 or 4 on 1–5 scale, with 4 being the most frequent.
- Very few rated 1 or 2, indicating low fatigue or low negative perception of energy.
- Overall pattern suggests participants generally feel energetic and active.

3 - 3 - 6. Visualization of key feature (Active sport) Distribution

```
# -- Distribution : Active sport --
col = "Active sport"
s = data[col].dropna()
plt.figure(figsize=(10, 5))
sns.histplot(s, bins=likert_bins, discrete=True, kde=False, stat="count")  # use stat="percent" for %
plt.xticks(likert_ticks)
plt.xlim(0.5, 5.5)
plt.xlabel(col)
plt.ylabel("Count")
plt.title(f"Distribution of {col} (Likert 1–5)")
plt.tight_layout()
plt.show()
```

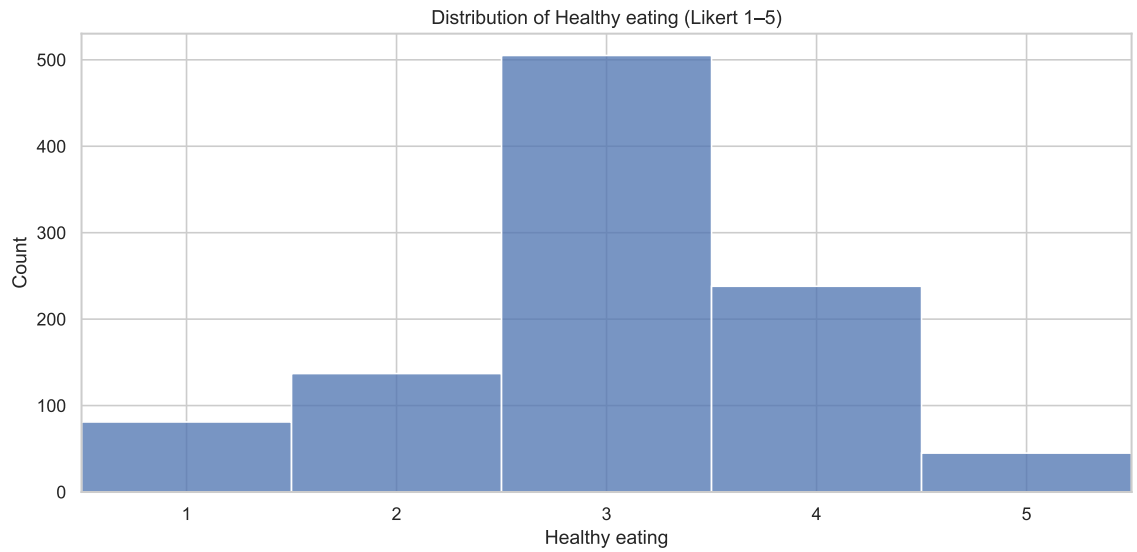


Small Comment for Distribution of Active sport

- Distribution of active sport participation shows bimodal pattern.
- Large group rated 1, meaning little or no physical activity, while another large group rated 5, indicating high activity.
- Middle values 2–4 less frequent, suggesting participants tend to be either inactive or highly active rather than moderate.

3 - 3 - 7. Visualization of key feature (Healthy eating) Distribution

```
# -- Distribution : Healthy eating --
col = "Healthy eating"
s = data[col].dropna()
plt.figure(figsize=(10, 5))
sns.histplot(s, bins=likert_bins, discrete=True, kde=False, stat="count")  # use stat="percent" for %
plt.xticks(likert_ticks)
plt.xlim(0.5, 5.5)
plt.xlabel(col)
plt.ylabel("Count")
plt.title(f"Distribution of {col} (Likert 1–5)")
plt.tight_layout()
plt.show()
```

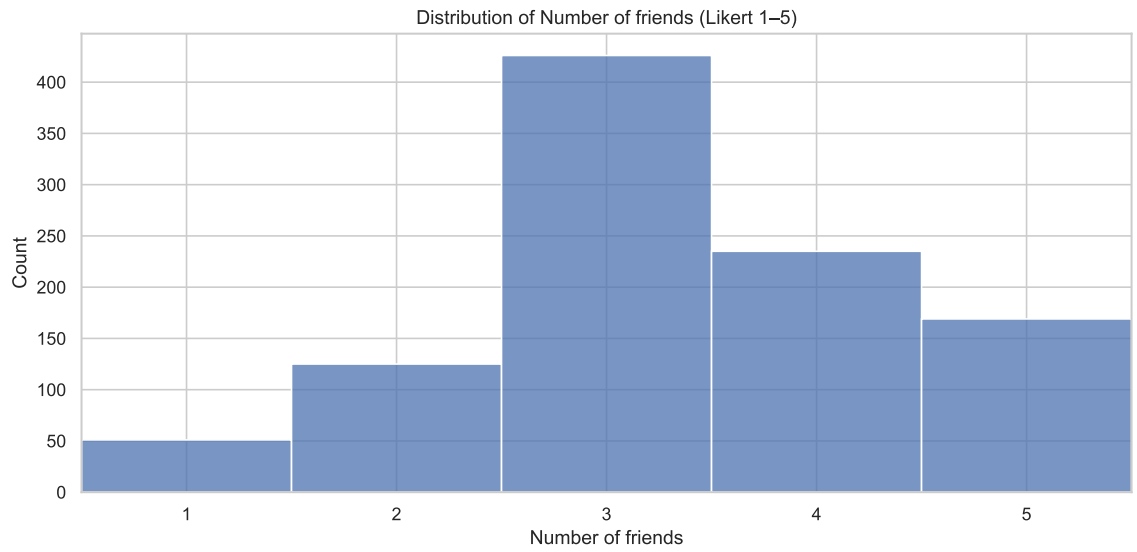


Small Comment for Distribution of Healthy eating

- Distribution of healthy eating shows strong concentration at 3, representing moderate dietary habits.
- Fewer participants rated 1 or 2, meaning unhealthy eating is relatively uncommon.
- Ratings of 4 or 5 are less frequent than 3, suggesting most respondents follow balanced but not strictly healthy diets.

3 - 3 - 8. Visualization of key feature (Number of friends) Distribution

```
# -- Distribution : Number of friends --
col = "Number of friends"
s = data[col].dropna()
plt.figure(figsize=(10, 5))
sns.histplot(s, bins=likert_bins, discrete=True, kde=False, stat="count")  # use stat="percent" for %
plt.xticks(likert_ticks)
plt.xlim(0.5, 5.5)
plt.xlabel(col)
plt.ylabel("Count")
plt.title(f"Distribution of {col} (Likert 1–5)")
plt.tight_layout()
plt.show()
```

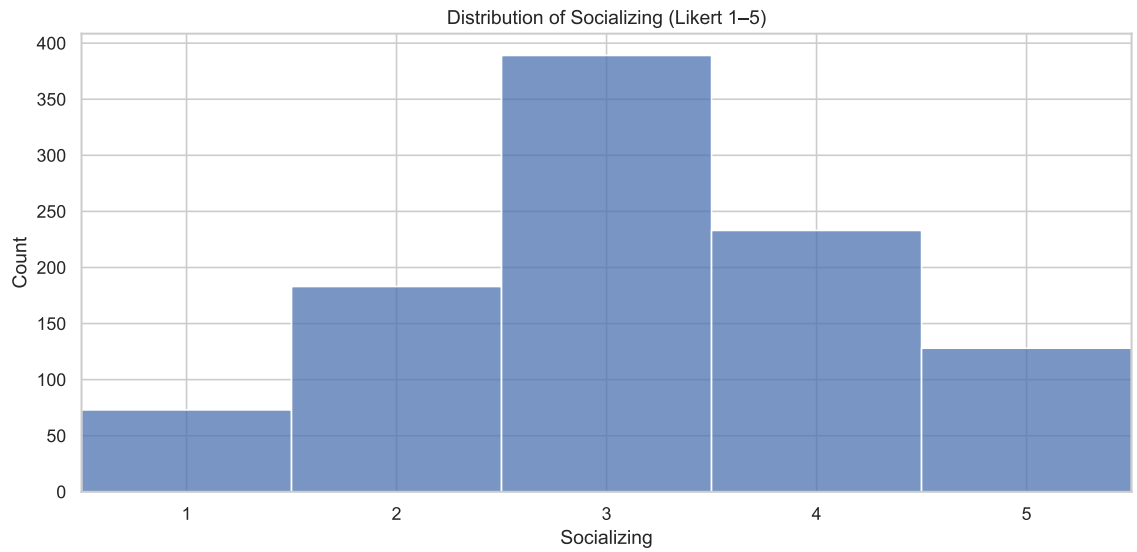


Small Comment for Distribution of Number of friends

- Distribution of number of friends peaks at 3, showing most participants have moderate social circles.
- Values 1 and 2 less common, meaning few report having very few friends.
- Ratings of 4 and 5 moderately frequent, indicating some participants maintain larger friend groups.
- Overall distribution suggests balanced but not extreme social connectivity.

3 - 3 - 9. Visualization of key feature (Socializing) Distribution

```
# -- Distribution : Socializing --
col = "Socializing"
s = data[col].dropna()
plt.figure(figsize=(10, 5))
sns.histplot(s, bins=likert_bins, discrete=True, kde=False, stat="count")  # use stat="percent" for %
plt.xticks(likert_ticks)
plt.xlim(0.5, 5.5)
plt.xlabel(col)
plt.ylabel("Count")
plt.title(f"Distribution of {col} (Likert 1–5)")
plt.tight_layout()
plt.show()
```

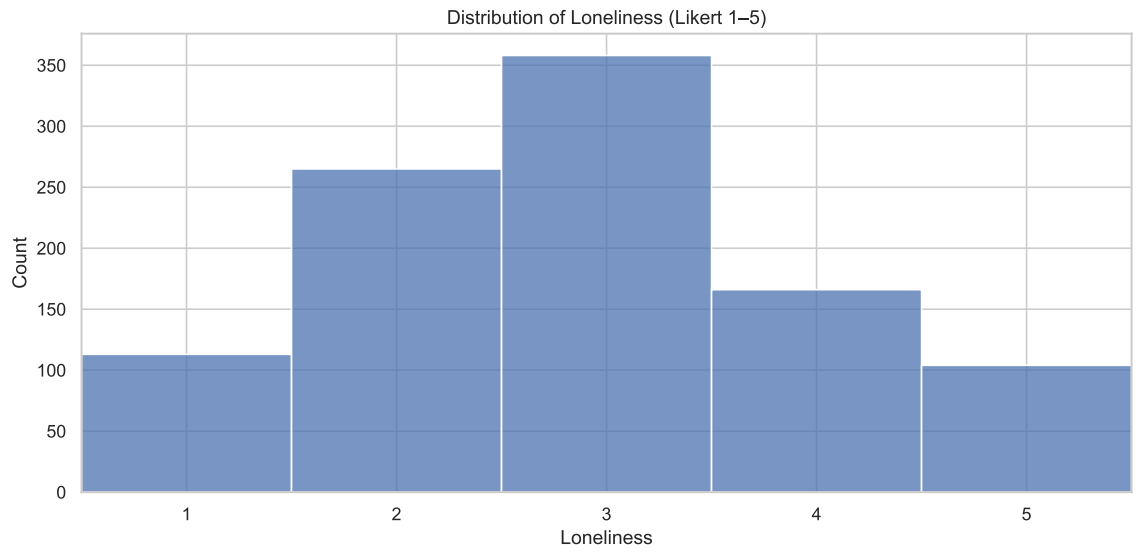


Small Comment for Distribution of Socializing

- Distribution of socializing frequency peaks at 3, showing majority of participants socialize at moderate level.
- Counts gradually decrease toward 1 and 5, meaning few are either very isolated or highly social.
- Overall pattern suggests balanced social engagement with moderate variation among individuals.

3 - 3 - 10. Visualization of key feature (Loneliness) Distribution

```
# -- Distribution : Loneliness --
col = "Loneliness"
s = data[col].dropna()
plt.figure(figsize=(10, 5))
sns.histplot(s, bins=likert_bins, discrete=True, kde=False, stat="count")  # use stat="percent" for %
plt.xticks(likert_ticks)
plt.xlim(0.5, 5.5)
plt.xlabel(col)
plt.ylabel("Count")
plt.title(f"Distribution of {col} (Likert 1–5)")
plt.tight_layout()
plt.show()
```

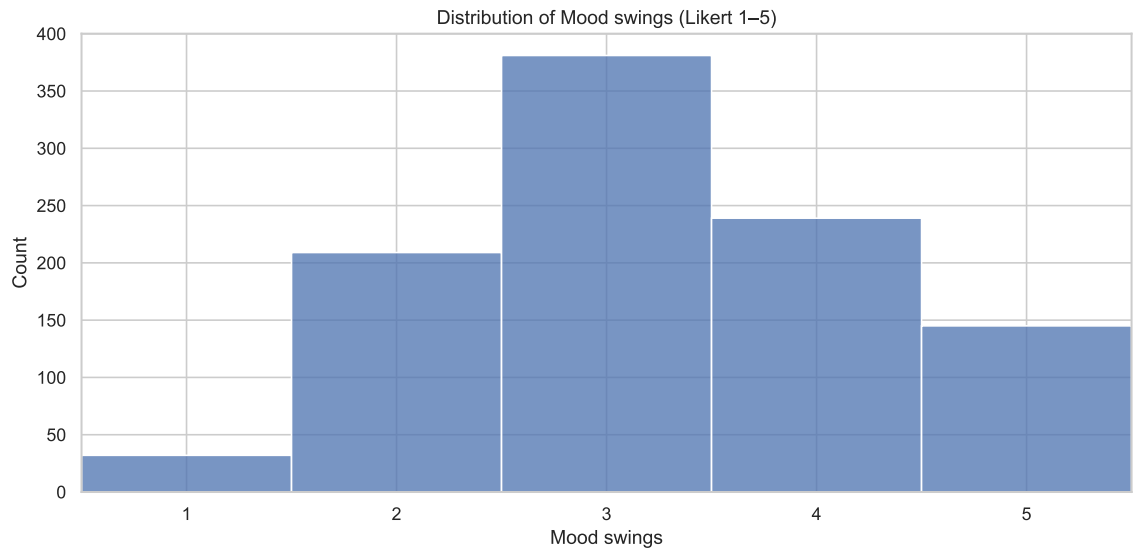


Small Comment for Distribution of Loneliness

- Distribution of loneliness centers at 3, showing most participants experience moderate loneliness.
- Values 1 and 2 occur relatively often, while 4 and 5 less frequent, suggesting fewer participants feel strong loneliness.
- Overall pattern indicates mild to moderate loneliness level among majority.

3 - 3 - 11. Visualization of key feature (Mood swings) Distribution

```
# -- Distribution : Mood swings --
col = "Mood swings"
s = data[col].dropna()
plt.figure(figsize=(10, 5))
sns.histplot(s, bins=likert_bins, discrete=True, kde=False, stat="count")  # use stat="percent" for %
plt.xticks(likert_ticks)
plt.xlim(0.5, 5.5)
plt.xlabel(col)
plt.ylabel("Count")
plt.title(f"Distribution of {col} (Likert 1–5)")
plt.tight_layout()
plt.show()
```

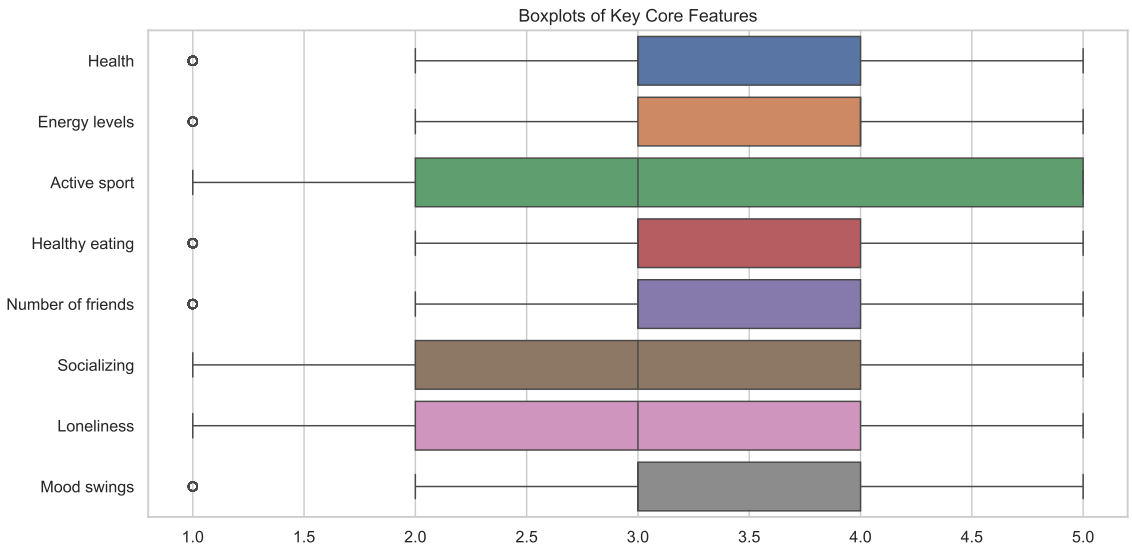


Small Comment for Distribution of Mood swings

- Distribution of mood swings peaks at 3, showing most participants report moderate emotional fluctuation.
- Ratings of 2 and 4 also common, indicating balanced variation between stable and unstable moods.
- Few rated 1 or 5, suggesting extreme emotional stability or instability is rare.
- Overall, emotional state appears mostly steady with moderate variability.

3 - 3 - 12. (Additional) Boxplot Analysis for key features

```
plot_features = [
    "Health", "Energy levels", "Active sport", "Healthy eating",
    "Number of friends", "Socializing", "Loneliness", "Mood swings"
]
plt.figure(figsize=(12, 6))
sns.boxplot(data=data[plot_features], orient="h")
plt.title('Boxplots of Key Core Features')
plt.show()
```



Interpretation for Boxplot Analysis of key features

Overall pattern
(1) Medians mostly in 3 to 4 band
(2) IQR around 1 Likert unit for most features meaning moderate variability
(3) Whiskers cover wide part of 1 to 5 scale for many features meaning small groups at extremes
(4) Few clear outliers meaning distributions fairly compact

Health
(1) Median near 3 to 4
(2) Slight positive tendency toward 4 to 5
(3) IQR moderate meaning diverse but not extreme self ratings

Energy levels
(1) Median close to 4
(2) IQR moderate and upper whisker long meaning many high energy respondents
(3) Few very low energy cases

Active sport
(1) Widest spread among features
(2) Median around 3 to 4 and long whiskers on both sides meaning polarization between inactive and very active groups
(3) Variability larger than other lifestyle variables

Healthy eating
(1) Median around 3
(2) IQR narrower than others meaning responses clustered near average eating habits
(3) High end ratings fewer than Energy levels

Number of friends
(1) Median around 3 to 4

(2) Balanced tails meaning mix of small and large social circles
(3) Variability moderate

Socializing
(1) Median near 4
(2) Right leaning pattern consistent with fairly active social life
(3) Minority with low socializing present

Loneliness
(1) Median at 3
(2) Upper tail shorter than lower tail meaning fewer highly lonely respondents
(3) Spread moderate

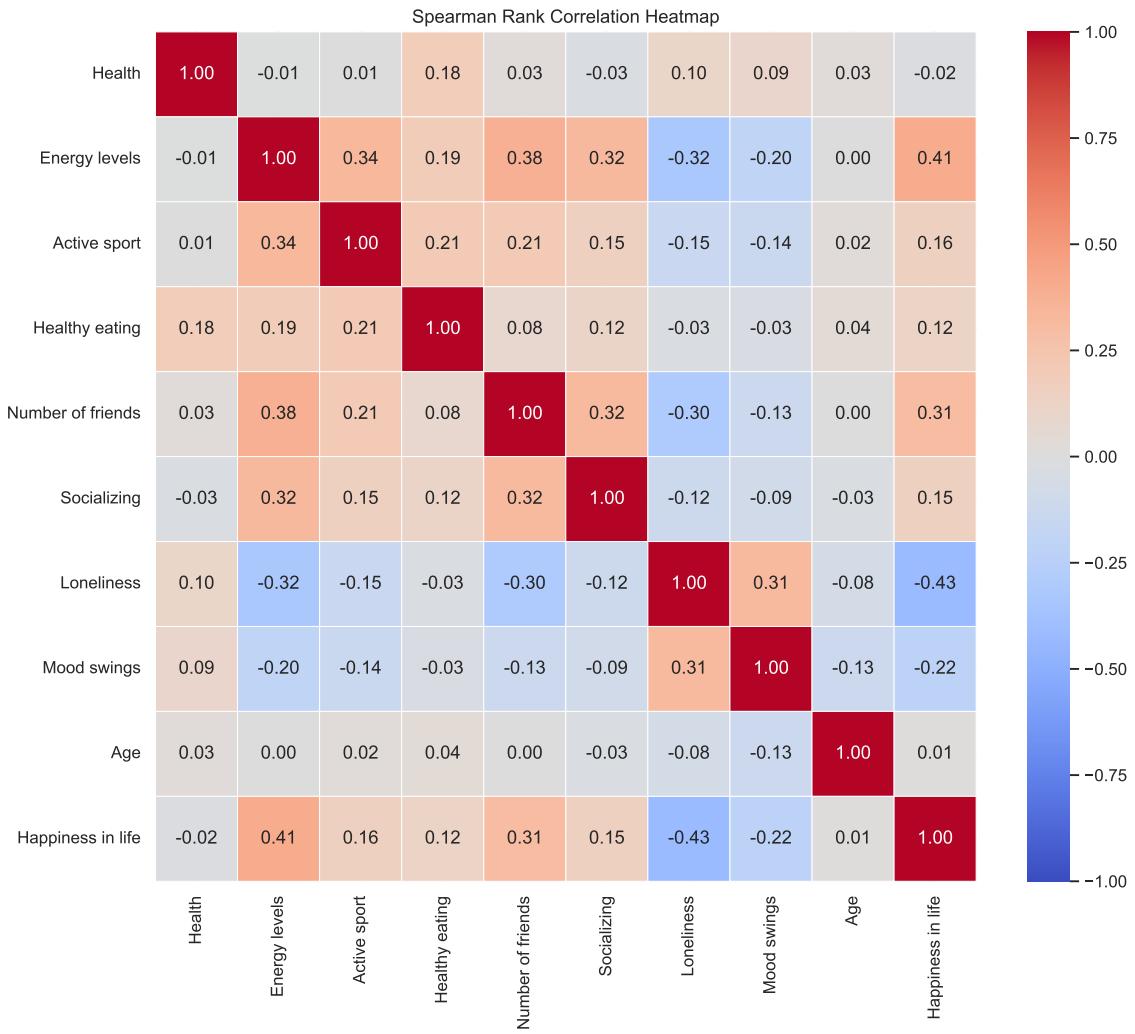
Mood swings
(1) Median around 3 to 4
(2) IQR moderate with roughly symmetric tails meaning moderate emotional variability for most
(3) Extremes rare

What is IQR ?

IQR means interquartile range, which represents spread of middle 50 percent of data. Q1 means 25th percentile that divides lower quarter from rest, and Q3 means 75th percentile that divides upper quarter. IQR equals Q3 minus Q1 and describes how wide central region of data is. In a boxplot, left edge of box corresponds to Q1, right edge to Q3, and line inside box to median. Whiskers usually extend to most extreme values that are within 1.5 times IQR from Q1 and Q3, while dots beyond whiskers represent outliers. Large IQR indicates high variability in main body of data, whereas small IQR means responses are tightly grouped around center.

3 - 4. Associations among numeric variables (Spearman correlation)

```
# Continuous-Continuous (Spearman Correlation)
corr_matrix = data[num_cols].corr(method='spearman')
plt.figure(figsize=(12, 10))
sns.heatmap(
    corr_matrix,
    annot=True,        # Show correlation values
    fmt=".2f",         # Format to 2 decimal places
    cmap='coolwarm',   # Color map
    vmin=-1, vmax=1,   # Fix range from -1 to 1
    linewidths=0.5
)
plt.title('Spearman Rank Correlation Heatmap')
plt.show()
```



Spearman correlation heatmap interpretation

- Strongest positive links with happiness in life appear for energy levels and number of friends
- Strongest negative link with happiness in life appears for loneliness
- Moderate positive links observed among lifestyle variables such as active sport and healthy eating
- Weak or near zero links indicate limited association between some pairs
- Heatmap helps spot clusters of related variables by warm colors and cold colors

3 - 5. Gender differences in personal traits and happiness

```
# Categorical-Continuous (Gender vs. Happiness)
# Used planned Mann-Whitney U test (non-parametric).

# keep only female and male
d = data[data["Gender"].isin(["female", "male"])].copy()

# groups
x = d.loc[d["Gender"] == "female", target].dropna()
y = d.loc[d["Gender"] == "male",   target].dropna()

# boxplot with sample sizes
plt.figure(figsize=(8, 5))
sns.boxplot(x="Gender", y=target, data=d)
plt.title(f"Happiness in life by Gender  (n_f={len(x)}, n_m={len(y)})")
plt.tight_layout()
plt.show()

# Mann-Whitney U (two-sided)
test_statistic, p_value = stats.mannwhitneyu(x, y, alternative="two-sided", method="auto")

# Effect size (Cliff's delta)
import numpy as np
from itertools import product
effect_size = (np.sum([xi > yi for xi, yi in product(x, y)]) -
               np.sum([xi < yi for xi, yi in product(x, y)])) / (len(x) * len(y))

n_female = len(x)
n_male = len(y)

print(f"U-statistic={test_statistic:.2f}, p-value={p_value:.4f}, Effect size (Cliff's delta)={effect_size:.3f}  (n_female={
```

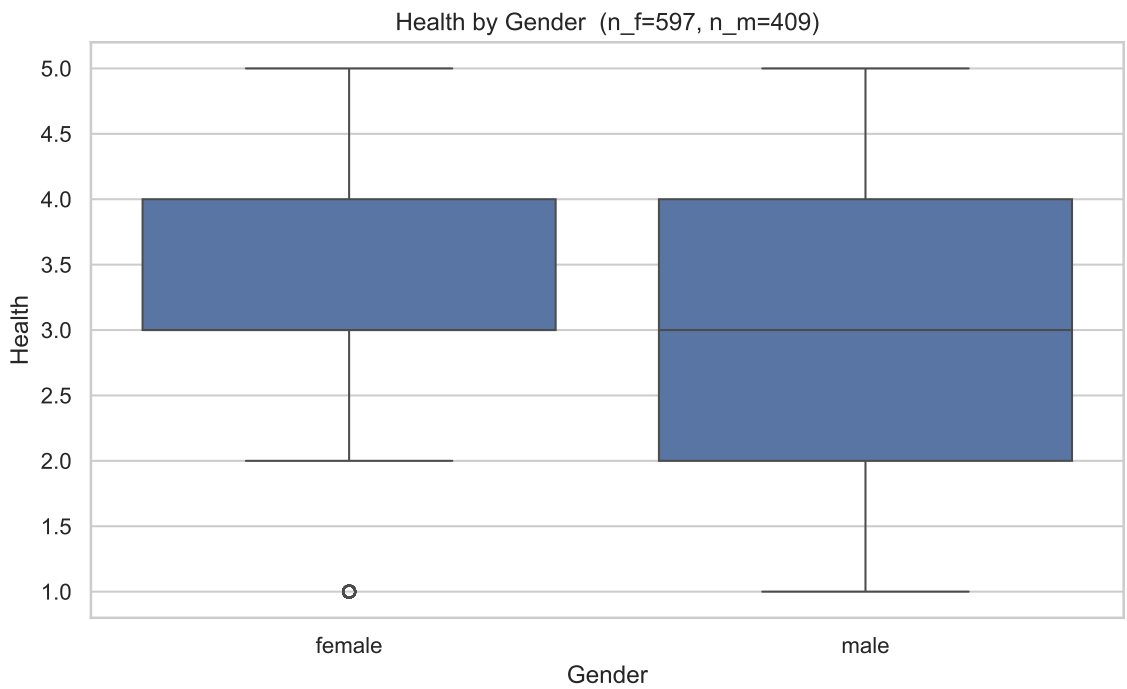
U-statistic=115634.50, p-value=0.1210, Effect size (Cliff's delta)=-0.053 (n_female=597, n_male=409)

Boxplot (Happiness in life) by gender interpretation

- Median happiness similar across female and male
- Interquartile range comparable which suggests similar spread
- Presence of similar whiskers indicates similar range of typical values
- Outliers appear at low end for both groups which suggests rare very low scores
- Test result guides conclusion about statistical significance beyond visual similarity

Based on interpretation rules above, following Boxplot by Gender continued from next page can also be interpreted easily !

```
var = "Health"
plt.figure(figsize=(8,5))
sns.boxplot(x="Gender", y=var, data=d)
plt.title(f"{var} by Gender  (n_f={len(x)}, n_m={len(y)})")
plt.tight_layout()
plt.show()

u, p = stats.mannwhitneyu(x, y, alternative="two-sided", method="auto")
delta = (np.sum([xi>yi for xi,yi in product(x,y)]) - np.sum([xi<yi for xi,yi in product(x,y)])) / (len(x)*len(y))
print(f"U-statistic={u:.2f}, p-value={p:.4f}, Effect size (Cliff's delta)={delta:.3f}  (n_female={len(x)}, n_male={len(y
```

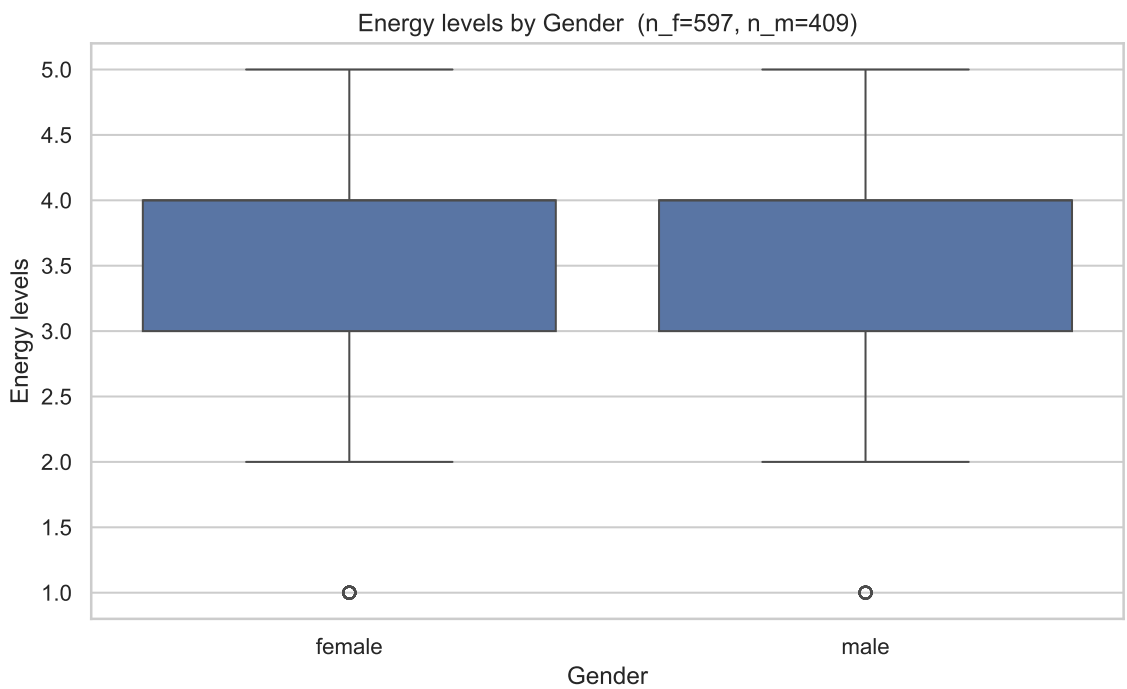


U-statistic=115634.50, p-value=0.1210, Effect size (Cliff's delta)=-0.053 (n_female=597, n_male=409)

Boxplot (Health) by gender interpretation

- Median health slightly higher for female than male
- Interquartile range wider for male showing greater variability
- Whiskers longer for male group meaning broader range of health perception
- Few low-end outliers in both genders representing small number with poor health
- Statistical test result with small effect size and nonsignificant p-value supports visual observation of minimal gender difference

```
var = "Energy levels"
plt.figure(figsize=(8,5))
sns.boxplot(x="Gender", y=var, data=d)
plt.title(f"{var} by Gender  (n_f={len(x)}, n_m={len(y)})")
plt.tight_layout()
plt.show()

u, p = stats.mannwhitneyu(x, y, alternative="two-sided", method="auto")
delta = (np.sum([xi>yi for xi,yi in product(x,y)]) - np.sum([xi<yi for xi,yi in product(x,y)])) / (len(x)*len(y))
print(f"U-statistic={u:.2f}, p-value={p:.4f}, Effect size (Cliff's delta)={delta:.3f}  (n_female={len(x)}, n_male={len(y
```

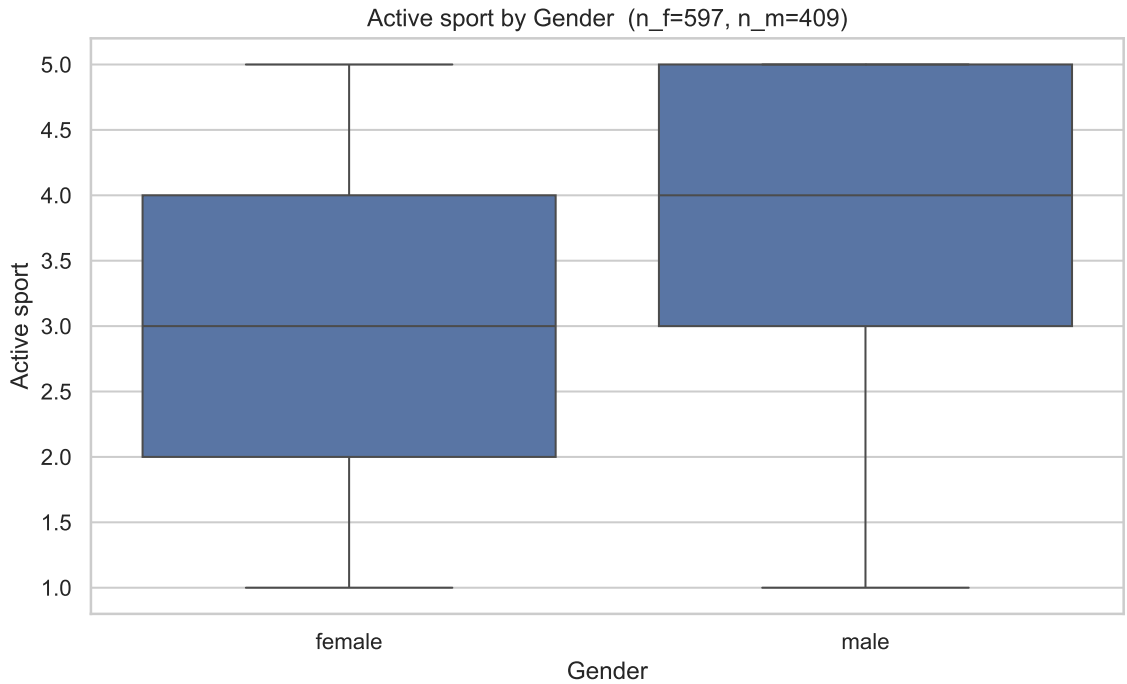


U-statistic=115634.50, p-value=0.1210, Effect size (Cliff's delta)=-0.053 (n_female=597, n_male=409)

Boxplot (Energy levels) by gender interpretation

- Median energy levels nearly identical for female and male
- Interquartile range similar between genders showing comparable variability
- Whiskers symmetric and extend to similar range meaning overall energy distribution alike
- One low-end outlier in each group indicating few participants with very low energy
- Statistical test with small effect size and nonsignificant p-value confirms no meaningful gender difference in energy perception

```
var = "Active sport"
plt.figure(figsize=(8,5))
sns.boxplot(x="Gender", y=var, data=d)
plt.title(f"{var} by Gender  (n_f={len(x)}, n_m={len(y)})")
plt.tight_layout()
plt.show()

u, p = stats.mannwhitneyu(x, y, alternative="two-sided", method="auto")
delta = (np.sum([xi>yi for xi,yi in product(x,y)]) - np.sum([xi<yi for xi,yi in product(x,y)])) / (len(x)*len(y))
print(f"U-statistic={u:.2f}, p-value={p:.4f}, Effect size (Cliff's delta)={delta:.3f}  (n_female={len(x)}, n_male={len(y
```

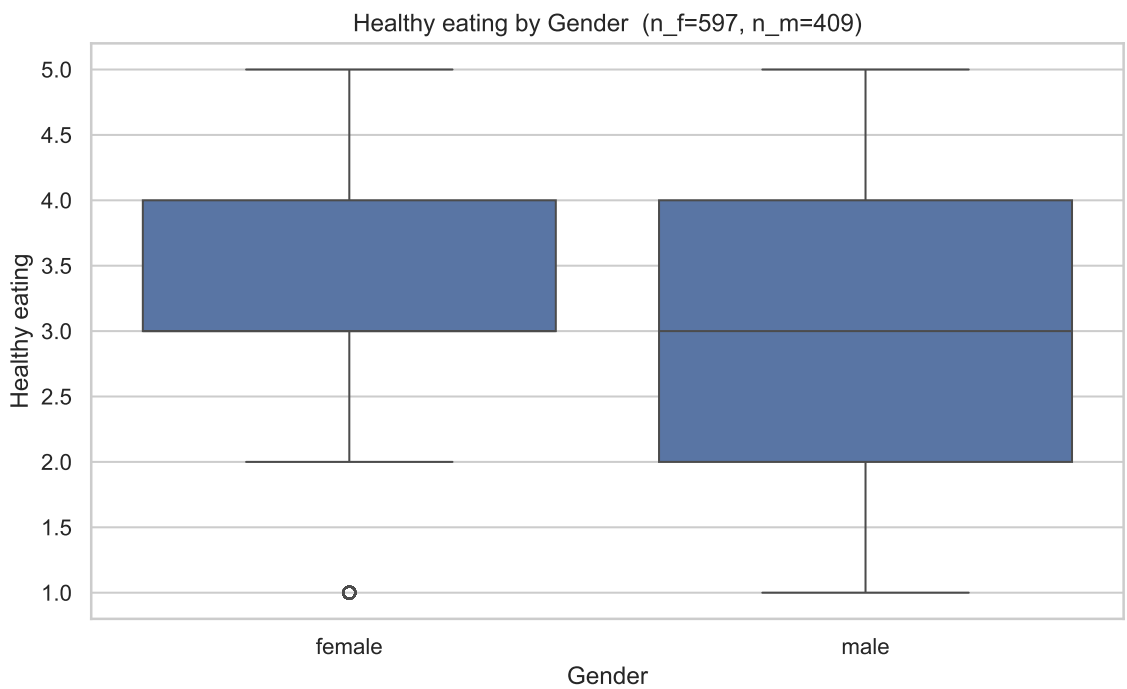


U-statistic=115634.50, p-value=0.1210, Effect size (Cliff's delta)=-0.053 (n_female=597, n_male=409)

Boxplot (Active sport) by gender interpretation

- Median active sport higher for male than female
- Interquartile range slightly wider for male showing greater diversity in activity level
- Whiskers for male extend fully from 1 to 5 indicating both inactive and highly active individuals
- Female group concentrated around 2 to 4 range suggesting moderate participation
- Statistical test with small effect size and nonsignificant p-value indicates difference not statistically meaningful though male group shows slightly higher activity tendency

```
var = "Healthy eating"
plt.figure(figsize=(8,5))
sns.boxplot(x="Gender", y=var, data=d)
plt.title(f"{var} by Gender  (n_f={len(x)}, n_m={len(y)})")
plt.tight_layout()
plt.show()

u, p = stats.mannwhitneyu(x, y, alternative="two-sided", method="auto")
delta = (np.sum([xi>yi for xi,yi in product(x,y)]) - np.sum([xi<yi for xi,yi in product(x,y)])) / (len(x)*len(y))
print(f"U-statistic={u:.2f}, p-value={p:.4f}, Effect size (Cliff's delta)={delta:.3f}  (n_female={len(x)}, n_male={len(y
```

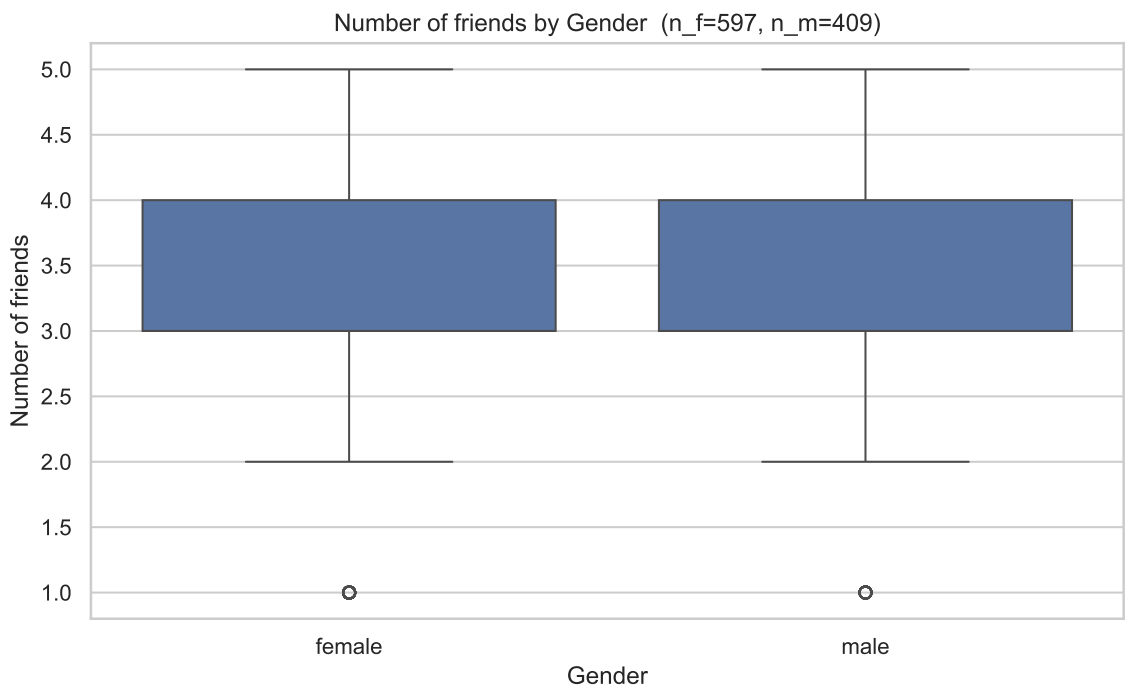


U-statistic=115634.50, p-value=0.1210, Effect size (Cliff's delta)=-0.053 (n_female=597, n_male=409)

Boxplot (Healthy eating) by gender interpretation

- Median healthy eating slightly higher for female than male
- Interquartile range broader for male suggesting wider variation in dietary habits
- Lower whisker for male reaches minimum value indicating presence of less healthy eaters
- Female group distribution tighter around 3 to 4 meaning more consistent eating behavior
- Statistical result with small effect size and nonsignificant p-value confirms no strong gender difference though females show slightly healthier tendency

```
var = "Number of friends"
plt.figure(figsize=(8,5))
sns.boxplot(x="Gender", y=var, data=d)
plt.title(f"{var} by Gender  (n_f={len(x)}, n_m={len(y)})")
plt.tight_layout()
plt.show()

u, p = stats.mannwhitneyu(x, y, alternative="two-sided", method="auto")
delta = (np.sum([xi>yi for xi,yi in product(x,y)]) - np.sum([xi<yi for xi,yi in product(x,y)])) / (len(x)*len(y))
print(f"U-statistic={u:.2f}, p-value={p:.4f}, Effect size (Cliff's delta)={delta:.3f}  (n_female={len(x)}, n_male={len(y
```

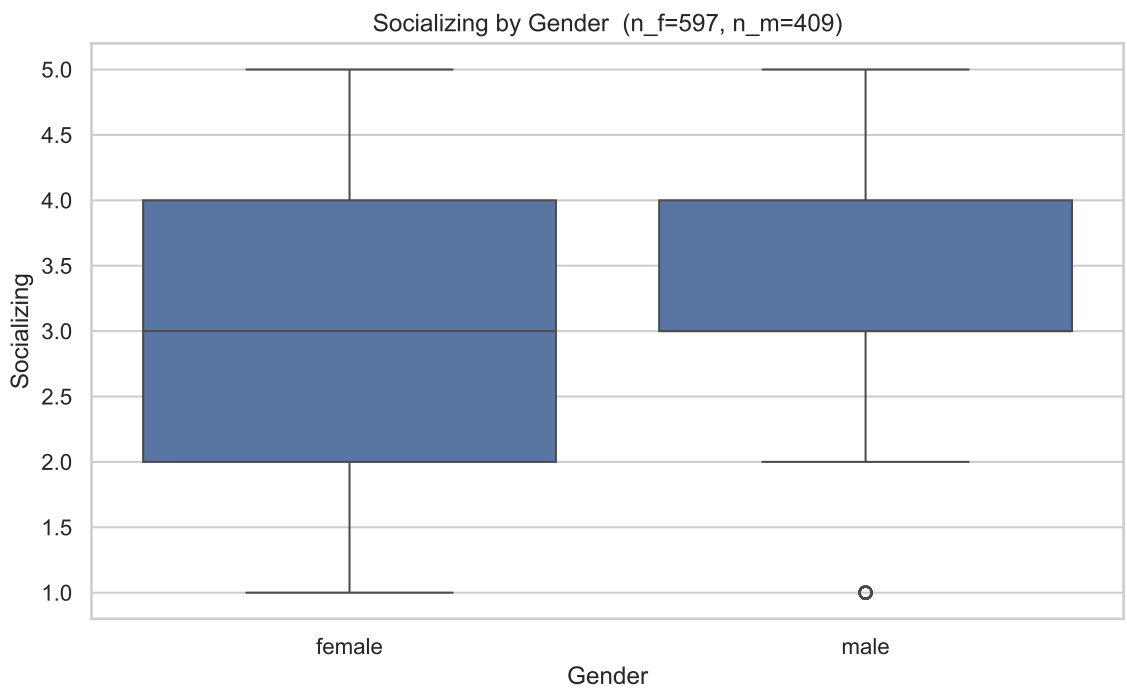


U-statistic=115634.50, p-value=0.1210, Effect size (Cliff's delta)=-0.053 (n_female=597, n_male=409)

Boxplot (Number of friends) by gender interpretation

- Median number of friends identical for female and male
- Interquartile ranges almost equal showing similar variability in social connections
- Whiskers extend to same limits with one low outlier in each group meaning few participants have very few friends
- Overall distribution shapes visually identical
- Statistical result with small effect size and nonsignificant p-value supports conclusion of no meaningful gender difference

```
var = "Socializing"
plt.figure(figsize=(8,5))
sns.boxplot(x="Gender", y=var, data=d)
plt.title(f"{var} by Gender  (n_f={len(x)}, n_m={len(y)})")
plt.tight_layout()
plt.show()

u, p = stats.mannwhitneyu(x, y, alternative="two-sided", method="auto")
delta = (np.sum([xi>yi for xi,yi in product(x,y)]) - np.sum([xi<yi for xi,yi in product(x,y)])) / (len(x)*len(y))
print(f"U-statistic={u:.2f}, p-value={p:.4f}, Effect size (Cliff's delta)={delta:.3f}  (n_female={len(x)}, n_male={len(y
```

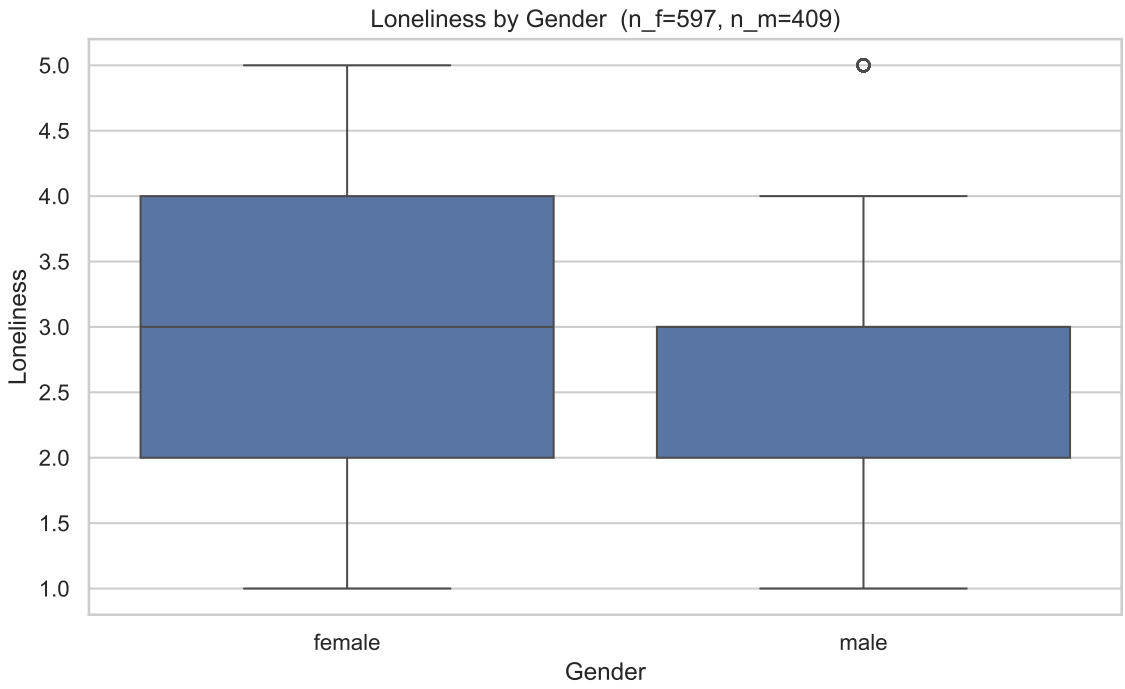


U-statistic=115634.50, p-value=0.1210, Effect size (Cliff's delta)=-0.053 (n_female=597, n_male=409)

Boxplot (Socializing) by gender interpretation

- Median socializing level same for female and male
- Interquartile ranges nearly equal showing similar spread of social activity
- Female group slightly more variable at lower end indicating few less social participants
- One mild low outlier in male group showing rare cases of very low social engagement
- Test result with small effect size and nonsignificant p-value indicates no substantial gender difference in socializing behavior

```
var = "Loneliness"
plt.figure(figsize=(8,5))
sns.boxplot(x="Gender", y=var, data=d)
plt.title(f"{var} by Gender  (n_f={len(x)}, n_m={len(y)})")
plt.tight_layout()
plt.show()

u, p = stats.mannwhitneyu(x, y, alternative="two-sided", method="auto")
delta = (np.sum([xi>yi for xi,yi in product(x,y)]) - np.sum([xi<yi for xi,yi in product(x,y)])) / (len(x)*len(y))
print(f"U-statistic={u:.2f}, p-value={p:.4f}, Effect size (Cliff's delta)={delta:.3f}  (n_female={len(x)}, n_male={len(y
```

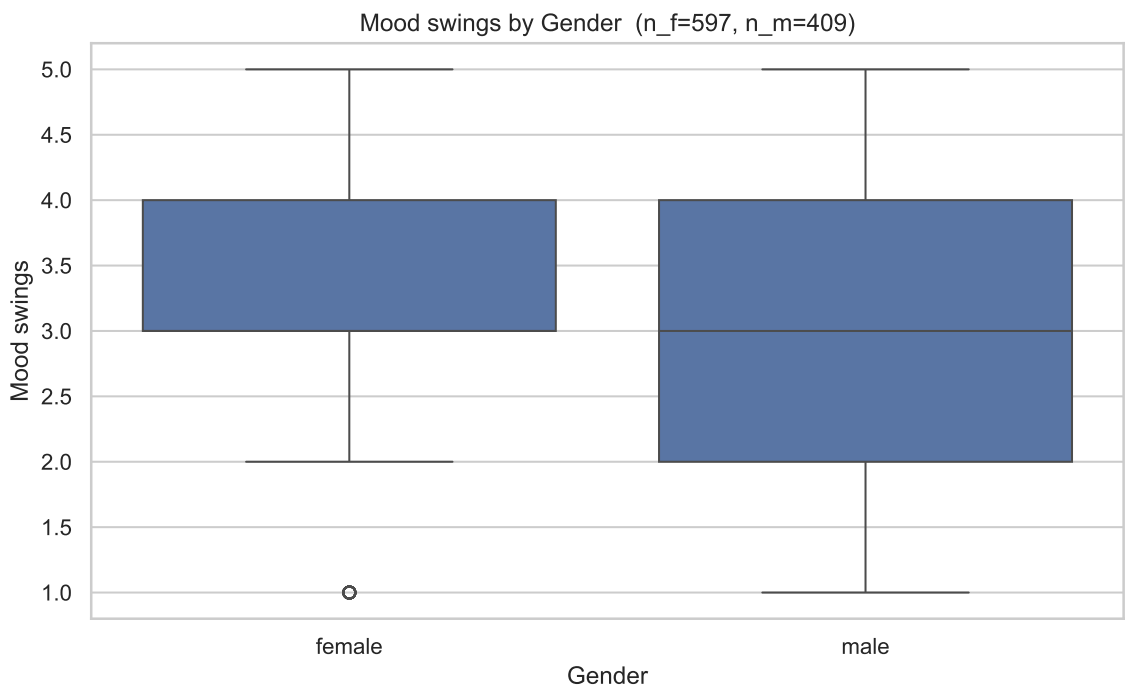


U-statistic=115634.50, p-value=0.1210, Effect size (Cliff's delta)=-0.053 (n_female=597, n_male=409)

Boxplot (Loneliness) by gender interpretation

- Median loneliness slightly higher for female than male
- Female group shows wider interquartile range meaning greater variation in perceived loneliness
- Male distribution more compact with one high outlier indicating a rare extreme loneliness case
- Overall spread similar but central tendency suggests females feel loneliness a bit more frequently
- Statistical test result with small effect size and nonsignificant p-value supports minimal practical gender difference

```
var = "Mood swings"
plt.figure(figsize=(8,5))
sns.boxplot(x="Gender", y=var, data=d)
plt.title(f"{var} by Gender  (n_f={len(x)}, n_m={len(y)})")
plt.tight_layout()
plt.show()

u, p = stats.mannwhitneyu(x, y, alternative="two-sided", method="auto")
delta = (np.sum([xi>yi for xi,yi in product(x,y)]) - np.sum([xi<yi for xi,yi in product(x,y)])) / (len(x)*len(y))
print(f"U-statistic={u:.2f}, p-value={p:.4f}, Effect size (Cliff's delta)={delta:.3f}  (n_female={len(x)}, n_male={len(y
```



U-statistic=115634.50, p-value=0.1210, Effect size (Cliff's delta)=-0.053 (n_female=597, n_male=409)

Boxplot (Mood swings) by gender interpretation

- Median mood swings nearly identical for female and male
- Male group exhibits slightly wider interquartile range meaning greater variability in emotional fluctuation
- Female group includes one low outlier representing rare case of very stable mood
- Whiskers extend similarly for both groups suggesting comparable overall range
- Statistical test result with small effect size and nonsignificant p-value confirms negligible gender difference in mood swings

Evaluation Metrics with Terminology Explanations

- **U-statistic** : numerical value that measures rank-based difference between two groups larger difference in rank sum means greater U value
- **p-value** : probability of observing result as extreme as this under null hypothesis small value (below 0.05) suggests statistically significant difference
- **Effect size (Cliff's delta)** : quantifies strength and direction of difference positive means higher values for females, negative means higher values for males ranges from −1 (completely opposite) to +1 (completely same order)
- **n_female** : number of valid observations in female group
- **n_male** : number of valid observations in male group

4. Modeling and Comparison

4 - 1. Data Splitting

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

from sklearn.model_selection import train_test_split, cross_validate, KFold
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, HuberRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.ensemble import RandomForestRegressor

# split
feature_cols = features_core + controls
X = data[feature_cols]
y = data[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Split : train {len(X_train)} | test {len(X_test)}")
```

Split : train 804 | test 202

Meaning !

After an 80/20 split, dataset resulted in 804 samples for training and 202 for testing.

4 - 2. Preprocessing Pipeline Construction

→ Numeric and categorical variables were separately processed using median imputation, scaling, and one-hot encoding.

```
# Preprocessing
numeric_features = [c for c in feature_cols if c in (features_core + ["Age"])]
categorical_features = [c for c in feature_cols if c in ["Gender"]]

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Dense OHE to avoid sparse-compat issues
ohe = OneHotEncoder(handle_unknown="ignore", drop="if_binary", sparse_output=False)

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", ohe)
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ],
    remainder="drop"
)
# Preprocessing pipeline ready !
```

4 - 3. Implementing THREE Regression Models (Linear, Huber, Random Forest)

→ Although the project requirement specified "Implement at least 2 different ML models," THREE models were implemented to achieve a higher level of completeness and quality in the project.

```
# Define models
models = {
    "Linear Regression": LinearRegression(),
    "Huber Regression": HuberRegressor(max_iter=500),
    "Random Forest": RandomForestRegressor(
        n_estimators=300,  # stable performance
        max_depth=None,    # let trees grow fully
        min_samples_leaf=1,
        random_state=42,
        n_jobs=-1
    ),
}
```

```python
# 5-fold CV with robust metrics (RMSE from MSE)
cv_results_list = []
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

for model_name, model_obj in models.items():
    full_pipeline = Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("regressor", model_obj)
    ])
    cv_scores = cross_validate(
        full_pipeline,
        X_train, y_train,
        cv=kfold,
        scoring=["neg_mean_absolute_error", "neg_mean_squared_error", "r2"],
        return_train_score=False
    )
    mae = -cv_scores["test_neg_mean_absolute_error"].mean()
    rmse = np.sqrt(-cv_scores["test_neg_mean_squared_error"].mean())
    r2 = cv_scores["test_r2"].mean()
    cv_results_list.append({"Model": model_name, "MAE (CV)": mae, "RMSE (CV)": rmse, "R² (CV)": r2})

cv_results_df = pd.DataFrame(cv_results_list).sort_values("MAE (CV)")
```

What I have done with TRHEE models : Summary

- Define THREE regression models: LinearRegression, HuberRegressor, and RandomForestRegressor
  - Using LinearRegression (simple linear regression) together with HuberRegressor (robust regression) corresponds exactly to the example provided in the task description
- Prepare 5-fold cross validation object to make evaluation more reliable
- For each model build same preprocessing plus regressor pipeline
- Run cross_validate on training data and collect three scores
  - neg_mean_absolute_error converted to MAE
  - neg_mean_squared_error converted to RMSE by square root
  - r2 as R^2 metric
- Compute mean of each metric across folds and store in list
- Convert results list to DataFrame and sort by MAE ascending
- Display table to compare both models in one view

```
print("\n--- Cross-Validation (CV) results ---")
display(cv_results_df.set_index("Model"))
```

--- Cross-Validation (CV) results ---

Model	MAE (CV)	RMSE (CV)	R^2 (CV)
Huber Regression	0.550961	0.701153	0.280520
Linear Regression	0.551979	0.700740	0.281095
Random Forest	0.573379	0.730966	0.219819

Interpretation of Results from above Table

- Both Huber Regression and Linear Regression show very similar performance
- MAE of Huber Regression (0.5509) is slightly lower, meaning slightly better average accuracy
- RMSE of Linear Regression (0.7007) is a bit lower, meaning slightly better handling of larger errors
- Random Forest shows higher MAE (0.5734) and RMSE (0.7310), indicating slightly worse predictive performance for this dataset
  - However, Random Forest still performs reasonably well and may capture nonlinear patterns that linear models cannot, though its advantage is not evident here
- R^2 around 0.28 for linear and Huber models indicates they explain about 28 percent of the variance in the target, while Random Forest explains slightly less (~0.22)
- Overall performance difference among the three models is small and results are almost identical
- Huber Regression remains the best choice, as it provides stable and robust predictions with minimal sensitivity to outliers

4 - 4. Best Model Selection

```
# Automatic best-model selection
best_mae = cv_results_df["MAE (CV)"].min()
candidates = cv_results_df[np.isclose(cv_results_df["MAE (CV)"], best_mae)]
if len(candidates) > 1:
    best_row = candidates.loc[candidates["RMSE (CV)"].idxmin()]
else:
    best_row = candidates.iloc[0]
best_model_name = best_row["Model"]
print(f"\nSelected best model by CV : {best_model_name}")
# Fit best model on train and evaluate on test
final_model_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", models[best_model_name])
])
final_model_pipeline.fit(X_train, y_train)
y_pred_test = final_model_pipeline.predict(X_test)
test_mae = mean_absolute_error(y_test, y_pred_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))
test_r2 = r2_score(y_test, y_pred_test)
print("\n--- Test Set Performance ---")
print(f"MAE  = {test_mae:.4f}")
print(f"RMSE = {test_rmse:.4f}")
print(f"R²   = {test_r2:.4f}")
```

Selected best model by CV : Huber Regression

--- Test Set Performance ---
MAE = 0.5429
RMSE = 0.6909
R^2 = 0.2453

Test Set Evaluation of the Best Model (Huber Regression)

- Huber Regression was automatically selected as the best model based on CV results
- MAE = 0.5429 means the model's average prediction error is about 0.54 units
- RMSE = 0.6909 is slightly higher than MAE indicating presence of some larger errors or mild outliers
- R^2 = 0.2453 shows the model explains about 24.5 percent of the variance in the target variable
- R^2 on the test set is a bit lower than CV average suggesting small generalization drop
- Overall the Huber model provides stable and robust predictions while being less sensitive to outliers

5. Interpretation and Limitations

In this chapter, analysis is conducted using Huber Regression Model selected as the best model in *previous* chapter, and its analytical results with interpretations and limitations are discussed in detail.

```
# Access fitted steps
preprocessor_step = final_model_pipeline.named_steps["preprocessor"]
regressor_step = final_model_pipeline.named_steps["regressor"]

# Rebuild feature names
num_feature_names = list(numeric_features)
ohe_step = preprocessor_step.named_transformers_["cat"].named_steps["onehot"]
cat_feature_names = list(ohe_step.get_feature_names_out(categorical_features))
all_feature_names = num_feature_names + cat_feature_names

# Coefficients table
coefs = np.asarray(regressor_step.coef_).ravel()
coef_df = pd.DataFrame({
    "Feature": all_feature_names,
    "Coefficient": coefs
}).sort_values("Coefficient", ascending=False)

print("\nCoefficients table")
print("# numeric features -> standardized per +1 SD")
print("# categorical one-hot -> effect vs reference level (not standardized)")
display(coef_df)
```

Coefficients table
numeric features -> standardized per +1 SD
categorical one-hot -> effect vs reference level (not standardized)

	Feature	Coefficient
1	Energy levels	0.204219
3	Healthy eating	0.105254
4	Number of friends	0.091007
9	Gender_male	0.078264
0	Health	0.027965
5	Socializing	0.004350
8	Age	-0.037891
2	Active sport	-0.058753
7	Mood swings	-0.063615
6	Loneliness	-0.254836

5 - 1. Basic concepts for Interpretation

- Coefficients Table shows numerical values representing how each feature affects predicted output in regression model
- **Numeric features : standardized per +1 SD** means numeric variables are standardized so that one unit change equals one standard deviation from mean
  - Each coefficient shows change in predicted value when feature increases by one SD
- **Categorical one hot effect vs reference level (not standardized)** means categorical variables are one hot encoded and each coefficient represents difference from reference category
  - These values are not standardized because they are binary indicators

5 - 2. Interpretation for Coefficients Table

- Positive coefficient means predicted happiness becomes higher when feature value increases or when category is active
- Negative coefficient means predicted happiness becomes lower when feature value increases or when category is active
- The strongest positive factor is Energy levels with coefficient 0.204
- Other positive factors are Healthy eating 0.105 and Number of friends 0.091
- Gender male 0.078 indicates higher predicted happiness for male compared to reference gender
- Small positive effects come from Health 0.028 and Socializing 0.004
- Negative factors include Age minus 0.038 Active sport minus 0.059 and Mood swings minus 0.064
- The strongest negative factor is Loneliness minus 0.255 which shows strong decrease in predicted happiness when loneliness is high
- Coefficient size shows relative importance inside this linear model not causal impact

5 - 3. Interpretation about Strongest Positive Association

```
# Strongest positive/negative/neutral drivers
K = 5
top_pos = coef_df.head(K)
top_neg = coef_df.tail(K).sort_values("Coefficient")  # most negative first

print("\nStrongest Positive Associations")
for f, v in zip(top_pos["Feature"], top_pos["Coefficient"]):
    print(f"- {f}: {v:.3f}")
```

Strongest Positive Associations
- Energy levels: 0.204
- Healthy eating: 0.105
- Number of friends: 0.091
- Gender_male: 0.078
- Health: 0.028

- Strongest positive associations show features with largest positive coefficients in the model.
- Energy levels show the strongest positive relationship with predicted happiness, meaning that people who feel more energetic tend to report higher happiness.
- Healthy eating and having a larger number of friends are also linked to greater predicted happiness, suggesting that maintaining healthy habits and social connections contributes positively.
- Gender male has a small positive effect, indicating that males are predicted to be slightly happier than the reference gender when other factors are held constant.
- Health shows a weak positive association, implying that better perceived health is modestly related to higher happiness.

5 - 4. Interpretation about Strongest Negative Association

```
print("\nStrongest Negative Associations")
for f, v in zip(top_neg["Feature"], top_neg["Coefficient"]):
    print(f"- {f}: {v:.3f}")
```

Strongest Negative Associations
- Loneliness: -0.255
- Mood swings: -0.064
- Active sport: -0.059
- Age: -0.038
- Socializing: 0.004

- Strongest negative associations show features with largest negative coefficients in the model.
- Loneliness has the strongest negative relationship with predicted happiness, showing that people who feel lonelier tend to be less happy.
- Mood swings and lower activity in sports are also associated with reduced happiness, reflecting emotional instability and lower physical engagement as potential negative factors.
- Older age slightly decreases predicted happiness, though the effect is small.

5 - 5. Interpretation about Neutral Association

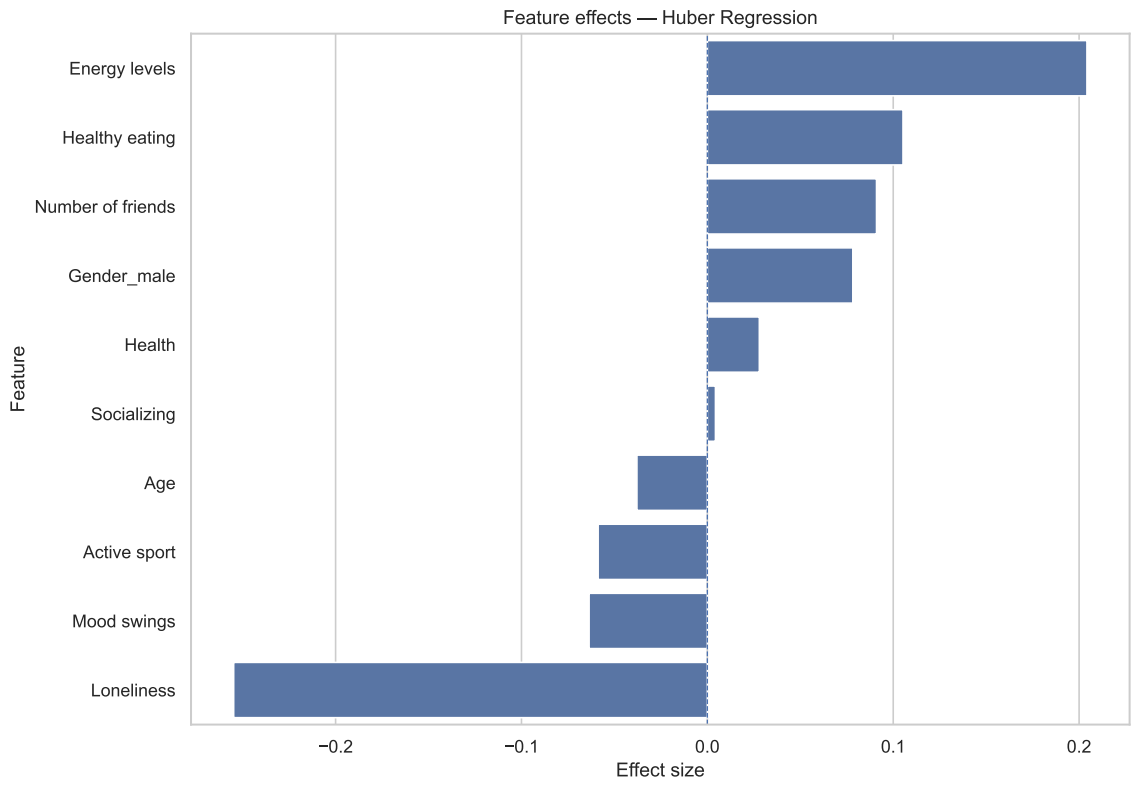
```
print("\nNeutral Feature")

neutral_feat = "Socializing"
neutral_row = coef_df[coef_df["Feature"] == neutral_feat]
coef_wo_neutral = coef_df[coef_df["Feature"] != neutral_feat]

if len(neutral_row) == 1:
    v = float(neutral_row["Coefficient"].iloc[0])
    print(f"- {neutral_feat}: {v:.3f}  (treated as neutral based on interpretation)")
else:
    print(f"- {neutral_feat}: not found in coefficient table")
```

Neutral Feature
- Socializing: 0.004 (treated as neutral based on interpretation)

- Socializing shows almost no effect, suggesting that the frequency of social interaction alone may not directly influence happiness in this model.

5 - 6. Effects of Features on Predicted Happiness using Huber Regression

```
# Bar plot of coefficients
plt.figure(figsize=(10, 7))
sns.barplot(x="Coefficient", y="Feature", data=coef_df)
plt.title(f"Feature effects — {best_model_name}")
plt.axvline(0.0, linestyle="--", linewidth=0.8)
plt.xlabel("Effect size")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

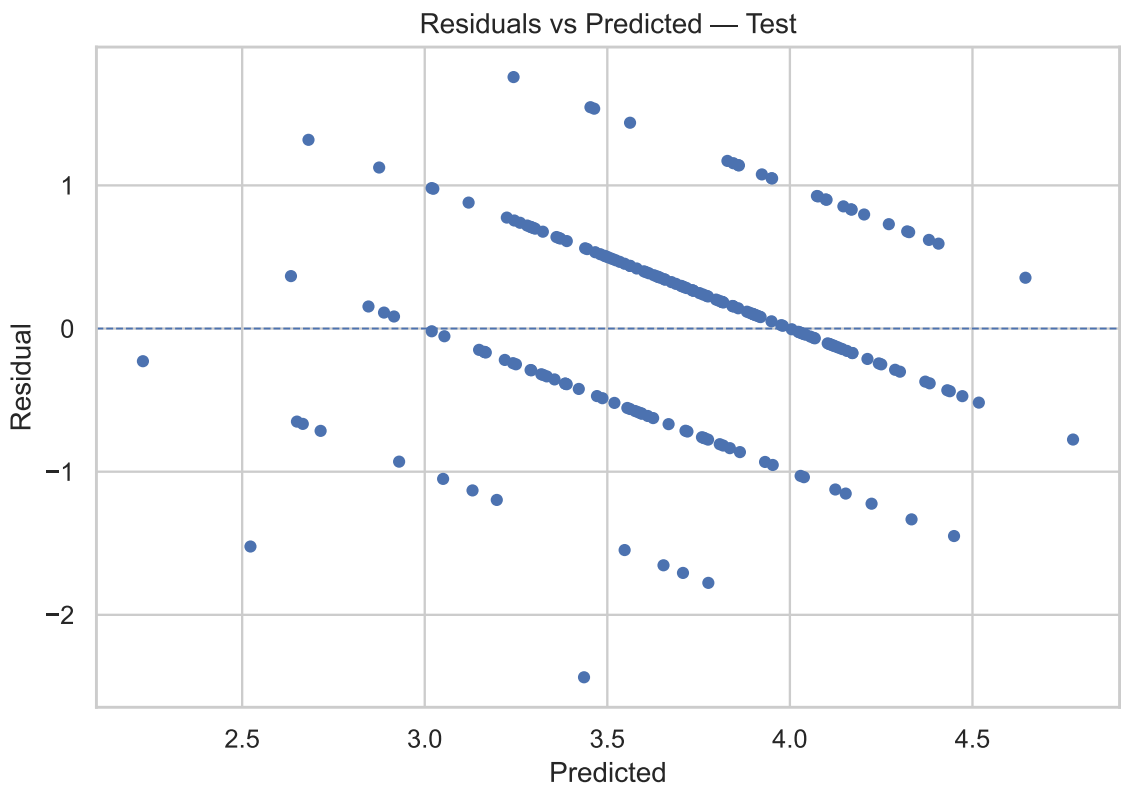


5 - 7. Interpretation of Above Bar Plot

- This bar plot visualizes how each feature contributes to predicted happiness score in Huber Regression model.
- Positive bars extend to right, meaning features are linked with higher predicted happiness.
- Negative bars extend to left, meaning features are linked with lower predicted happiness.
- Energy levels, healthy eating, number of friends, and being male show strong positive effects, indicating that people who feel energetic, eat healthily, and maintain social connections tend to report higher happiness.
- In contrast, loneliness, mood swings, and lower activity in sports have strong negative effects, suggesting that isolation, emotional instability, and inactivity decrease predicted happiness.
- Socializing and age have coefficients close to zero, meaning they have almost no direct linear effect in this model.
- Overall, plot shows which features most strongly influence model predictions and whether their impact is positive or negative.

5 - 8. Residuals vs Predicted on Test Set using Huber Regression

```
# Residual check on test set
residuals = y_test - y_pred_test
plt.figure(figsize=(7, 5))
plt.scatter(y_pred_test, residuals, s=18)
plt.axhline(0.0, linestyle="--", linewidth=0.8)
plt.xlabel("Predicted")
plt.ylabel("Residual")
plt.title("Residuals vs Predicted — Test")
plt.tight_layout()
plt.show()
```



5 - 9. Interpretation of Above Residual Plot

- This residual plot shows relationship between predicted values and residuals on test set for Huber Regression model.
- Points are roughly centered around zero line, which indicates unbiased errors on average.
- Diagonal bands appear because target is Likert scale with few discrete levels and residual equals true value minus prediction, so each level forms line with slope minus one.

- No strong curved pattern is visible, so linear specification is broadly adequate.
- Spread of residuals is moderate and fairly similar across prediction range, suggesting no severe heteroscedasticity, though variance looks slightly larger in mid range.
- A few points lie beyond minus two and plus two, which indicates outliers but count is small, consistent with Huber robustness.
- Positive coefficient means higher predicted target when feature increases or when category is active.
- Negative coefficient means lower predicted target under same interpretation.
- Categorical effect represents difference relative to dropped reference level since binary category was encoded with drop option.

5 - 10. Overall Conclusion

Analysis based on Huber Regression Model reveals key lifestyle and emotional factors associated with self-reported happiness among young people.
Energy levels emerge as the strongest positive driver, followed by healthy eating and number of friends, showing that physical vitality, balanced diet, and social connections contribute positively to happiness. ***Gender (male)*** also shows a mild positive association, while perceived health has smaller supportive effect.

On the other hand, ***loneliness*** exhibits the strongest negative impact on predicted happiness, followed by mood swings, reduced physical activity, and slightly older age, all of which reflect emotional instability and social isolation as dominant negative influences.***Socializing*** frequency shows nearly neutral effect, suggesting that quality of interaction may matter more than quantity.

Residual diagnostics confirm that model errors are roughly unbiased and variance is stable across prediction range, supporting adequacy of linear specification.

However, moderate R^2 value indicates that many influential factors, such as personality traits, life circumstances, or socioeconomic context, remain unobserved.

Overall, Huber Regression provides robust and interpretable results while mitigating influence of outliers, allowing balanced insight into how health, behavior, and emotion jointly relate to happiness.

5 - 11. Model-related Limitations

- Huber regression downweights extreme observations, which may hide valid but rare patterns.
- Multicollinearity between predictors can make coefficients unstable.
- Model assumes linear relationships and equal variance of errors, so residual diagnostics are required to confirm validity.
- Moderate R^2 indicates limited explanatory power, suggesting that additional predictors may be needed.

5 - 12. Dataset-related limitations

- Self-report bias possible since survey responses rely on participant honesty.
- Likert 1–5 scale treated as continuous, so equal-interval assumption may not hold.
- Cross-sectional data prevent causal interpretation between variables.
- Median or mode imputation may reduce variance and slightly bias estimates.

6. Epliogue : Thoughts I Had During Project

This project aimed to identify key lifestyle and emotional factors explaining self-reported happiness among young people. Results from Huber Regression analysis show that higher energy levels, healthy eating, and strong social ties enhance happiness, while loneliness and emotional instability have most negative effects. This is fundamental summary of this project in terms of research question and analytical results. Below, I want to share my **philosophy** behind this work.

Most important goal for me throughout this project was to get familiar with programming language Python. I'm pretty sure that I demonstrated my adaptability in Python. Second goal was to write reader-friendly report. I set this as my main philosophy. Even in daily conversations, people often ignore listeners and just speak. I believe such behavior is egocentric. Same applies to report writing. Some people write without caring about readers. I didn't want to be one of those authors.

That is why I created detailed Table of Contents, invested much time, and went through many trials and errors to design clear tables, draw underlines, make letters bold or italic, and color texts in various ways (just like I'm doing right now in this text). ***Not just throwing plots to readers but writing interpretations with them always***. These are techniques I used for reader-friendly report.

In terms of file format, I decided not to write report in Jupyter Notebook (.ipynb), but in Quarto (.qmd), which is extended format of R Markdown. Of course, Jupyter Notebook is excellent tool for executing Python codes, and no one can easily deny that. **But what about documentation?** Imagine someone with no background in computer science or programming receives Jupyter Notebook file with codes and short comments, believing it to be proper document, and even prints it out. At moment they see printed result, they would probably feel disappointed. People usually resist unfamiliar formats and need time to accept them. In my opinion, for general audiences, for example my grandmother, Jupyter Notebook is still far from being accessible document format. Unfortunately! Therefore, I decided to use Quarto, Markdown system specialized in documentation, which allows not only ***code execution*** but also creation of ***beautifully formatted and familiar documents***. Result is this report. As graduate student from Computer Science faculty, I would say that Jupyter Notebook is mainly tool for typical coding geeks 'yet'.

Finally, in Chapter 1–2, Data Summary Table, readers might have been overwhelmed by large table containing all variables. But I wanted this report to reflect my thought process, to show how I gradually observed entire dataset and built project step by step in natural flow. Many students studying data analysis tend to focus on flashy machine learning techniques, but my view is slightly different. ***I personally believe that taking broad view of dataset, making careful variable selection, and performing meticulous data preprocessing form true foundation of data science, even more important than fancy Machine Learning methods***. From this perspective, I devoted significant effort and space not only to methodology (I even applied THREE methods, exceeding requirement of two!) but also to Chapters 1 (Data Overview), 2 (Research Question and Variable Selection), and 3 (Exploratory Data Analysis), to emphasize importance of these fundamental steps.